

Progetto BIG DATA

Social network universitario basato su database a grafo

Introduzione

Nell'ambito dei sistemi informativi moderni, la gestione dei dati non riguarda soltanto la memorizzazione delle informazioni, ma anche la possibilità di analizzare le relazioni esistenti tra esse. In molti contesti reali, infatti, i dati non sono isolati, ma sono collegati tra loro attraverso rapporti diretti e indiretti.

Il presente progetto ha l'obiettivo di progettare e implementare un sistema per rappresentare un social network universitario, basato sulle relazioni sociali e accademiche tra studenti, docenti, corsi e gruppi di studio.

A differenza di un database relazionale tradizionale, che organizza i dati principalmente in tabelle, in questo progetto viene utilizzato un modello a grafo. In un database a grafo, le informazioni vengono rappresentate tramite nodi e relazioni. I nodi rappresentano le entità principali del sistema, come studenti, docenti, corsi e gruppi di studio, mentre le relazioni rappresentano i collegamenti tra queste entità.

L'utilizzo di un database a grafo risulta particolarmente adatto a questo caso, perché permette di analizzare facilmente connessioni tra persone, reti di collaborazione, studenti centrali nei gruppi e collegamenti indiretti tra soggetti diversi.

Il progetto prevede l'utilizzo di Docker per l'avvio dell'ambiente di lavoro, Neo4j come database a grafo e Python per interagire con il database ed eseguire alcune analisi sui dati.

Analisi dei requisiti

La prima fase della progettazione consiste nell'analisi dei requisiti. Questa fase ha lo scopo di comprendere quali informazioni devono essere memorizzate all'interno del sistema e quali operazioni il sistema deve permettere di eseguire.

Nel caso in esame, l'università vuole rappresentare le relazioni sociali e accademiche tra studenti, docenti, corsi e gruppi di studio. Poiché l'obiettivo principale non è soltanto archiviare i dati, ma anche analizzare i collegamenti tra le diverse entità, si è scelto di utilizzare un modello a grafo.

Il sistema dovrà quindi memorizzare le informazioni relative agli studenti, ai docenti, ai corsi universitari e ai gruppi di studio. Inoltre, dovrà rappresentare le relazioni tra questi elementi, come la frequenza dei corsi da parte degli studenti, l'insegnamento dei corsi da parte dei docenti, la partecipazione ai gruppi di studio, le conoscenze tra studenti e le interazioni tra studenti e docenti.

L'uso di un database a grafo consente di analizzare in modo più semplice le connessioni dirette e indirette tra le persone, individuando reti di collaborazione, studenti centrali nei gruppi e docenti maggiormente collegati agli studenti.

★ requisiti informativi:

il database a grafo deve memorizzare le seguenti informazioni:

- ❖ gestione degli studenti
 - per ogni studente devono essere registrati
 - matricola
 - nome
 - cognome
 - corso di laurea

- anno di iscrizione

esempio:

studente 1:

- matricola: 20066579
- nome: Luca
- cognome: Veraglia
- corso di laurea: Data Science
- anno di iscrizione: 2024

studente 2:

- matricola: 20088965
- nome: Marta
- cognome: De Stradis
- corso di laurea: Ingegneria Informatica
- anno di iscrizione: 2024

studente 3:

- matricola: 20029863
- nome: Patrizia
- cognome: Triponea
- corso di laurea: Ingegneria dell'Informazione
- anno di iscrizione: 2023

studente 4:

- matricola: 20016894
- nome: Paolo
- cognome: Riccardini
- corso di laurea: Sociologia
- anno di iscrizione: 2025

studente 5:

- matricola: 20073421
- nome: Sara
- cognome: Romano
- corso di laurea: Data Science
- anno di iscrizione: 2023

studente 6:

- matricola: 20054782
- nome: Davide
- cognome: Greco
- corso di laurea: Ingegneria Informatica
- anno di iscrizione: 2022

studente 7:

- matricola: 20091234
- nome: Elena
- cognome: Moretti
- corso di laurea: Economia
- anno di iscrizione: 2024

studente 8:

- matricola: 20045671
- nome: Marco
- cognome: Santoro
- corso di laurea: Sociologia
- anno di iscrizione: 2023

studente 9:

- matricola: 20082345
- nome: Chiara
- cognome: Ferri
- corso di laurea: Data Science
- anno di iscrizione: 2025

studente 10:

- matricola: 20037519
- nome: Andrea
- cognome: Rizzo
- corso di laurea: Ingegneria dell'Informazione
- anno di iscrizione: 2024

studente 11:

- matricola: 20069124

- nome: Giulia
- cognome: Conti
- corso di laurea: Lingue
- anno di iscrizione: 2023
- studente 12:
 - matricola: 20028467
 - nome: Simone
 - cognome: De Luca
 - corso di laurea: Economia
 - anno di iscrizione: 2022
- studente 13:
 - matricola: 20075643
 - nome: Francesca
 - cognome: Marino
 - corso di laurea: Psicologia
 - anno di iscrizione: 2024
- studente 14:
 - matricola: 20093418
 - nome: Matteo
 - cognome: Leone
 - corso di laurea: Ingegneria Informatica
 - anno di iscrizione: 2025
- studente 15:
 - matricola: 20051276
 - nome: Alessia
 - cognome: Costa
 - corso di laurea: Data Science
 - anno di iscrizione: 2022
- studente 16:
 - matricola: 20084329
 - nome: Federico
 - cognome: Caruso
 - corso di laurea: Statistica
 - anno di iscrizione: 2023
- studente 17:
 - matricola: 20036785
 - nome: Valentina
 - cognome: Gallo
 - corso di laurea: Lingue
 - anno di iscrizione: 2024
- studente 18:
 - matricola: 20097831
 - nome: Nicola
 - cognome: Esposito
 - corso di laurea: Sociologia
 - anno di iscrizione: 2022
- studente 19:
 - matricola: 20062145
 - nome: Martina
 - cognome: Serra
 - corso di laurea: Economia
 - anno di iscrizione: 2025
- studente 20:
 - matricola: 20048902
 - nome: Roberto
 - cognome: Longo
 - corso di laurea: Statistica
 - anno di iscrizione: 2024

❖ gestione dei docenti

per ogni docente devono essere registrati:

- nome

- cognome
- settore disciplinare
- email

esempio:

docente 1:

- nome: Gianluca
- cognome: Bellini
- settore disciplinare: Informatica
- email: Gianluca.bellini@unisalento.it

docente 2:

- nome: Alessandro
- cognome: Rinaldi
- settore disciplinare: Lingue
- email: Alessandro.rinaldi@unisalento.it

docente 3:

- nome: Giulia
- cognome: Ferraro
- settore disciplinare: Statistica e Matematica
- email: Giulia.ferraro@unisalento.it

docente 4:

- nome: Martina
- cognome: De Santis
- settore disciplinare: Internet
- email: Martina.desantis@unisalento.it

docente 5:

- nome: Paolo
- cognome: Marchetti
- settore disciplinare: Machine Learning
- email: paolo.marchetti@unisalento.it

docente 6:

- nome: Laura
- cognome: Conti
- settore disciplinare: Economia digitale
- email: laura.conti@unisalento.it

docente 7:

- nome: Stefano
- cognome: Greco
- settore disciplinare: Sociologia
- email: stefano.greco@unisalento.it

docente 8:

- nome: Elena
- cognome: Romano
- settore disciplinare: Basi di dati e sistemi informativi
- email: elena.romano@unisalento.it

- ❖ gestione dei corsi
- per ogni corso universitario devono essere registrati:
- codice del corso
 - nome del corso
 - numero di crediti

esempio:

corso 1:

- codice del corso: C001
- nome del corso: Internet of Things
- numero di crediti: 9

corso 2:

- codice del corso: B089
- nome del corso: analisi dei dati e big data
- numero di crediti: 8

corso 3:

- codice del corso: A290
- nome del corso: statistica
- numero di crediti: 8
- corso 4:
 - codice del corso: C008
 - nome del corso: lingua inglese
 - numero di crediti: 6
- corso 5:
 - codice del corso: A390
 - nome del corso: analisi matematica
 - numero di crediti: 9
- corso 6:
 - codice del corso: B120
 - nome del corso: Machine Learning
 - numero di crediti: 9
- corso 7:
 - codice del corso: C115
 - nome del corso: Basi di Dati
 - numero di crediti: 8
- corso 8:
 - codice del corso: E210
 - nome del corso: Economia Digitale
 - numero di crediti: 6
- corso 9:
 - codice del corso: S310
 - nome del corso: Sociologia delle Reti
 - numero di crediti: 6
- corso 10:
 - codice del corso: P205
 - nome del corso: Psicologia Sociale
 - numero di crediti: 8
- corso 11:
 - codice del corso: D330
 - nome del corso: Programmazione Python
 - numero di crediti: 9
- corso 12:
 - codice del corso: M410
 - nome del corso: Data Visualization
 - numero di crediti: 6

il corso rappresenta un insegnamento universitario. gli studenti possono seguire uno o più corsi e i docenti possono insegnare ad uno o più corsi.

- ❖ gestione dei gruppi di studio
per ogni gruppo di studio devono essere registrati:

- codice identificativo del gruppo
- nome del gruppo
- insegnamento a cui il gruppo è dedicato

esempio:

- gruppo 1:
 - codice identificativo del gruppo: G009
 - nome del gruppo: Gruppo Big Data
 - insegnamento a cui il gruppo è dedicato: big data
- gruppo 2:
 - codice identificativo del gruppo: G0863
 - nome del gruppo: Gruppo statistica & matematica
 - insegnamento a cui il gruppo è dedicato: Statistica e analisi matematica

gruppo 3:

- codice identificativo del gruppo: G0108
- nome del gruppo: internet
- insegnamento a cui il gruppo è dedicato: internet of things

gruppo 4:

- codice identificativo del gruppo: G6409
- nome del gruppo: gruppo studio inglese
- insegnamento a cui il gruppo è dedicato: lingua inglese

gruppo 5:

- codice identificativo del gruppo: G0215
- nome del gruppo: Gruppo Machine Learning
- insegnamento a cui il gruppo è dedicato: Machine Learning

gruppo 6:

- codice identificativo del gruppo: G0320
- nome del gruppo: Gruppo Database e Python
- insegnamento a cui il gruppo è dedicato: Basi di Dati e Programmazione Python

gruppo 7:

- codice identificativo del gruppo: G0477
- nome del gruppo: Gruppo Economia e Reti
- insegnamento a cui il gruppo è dedicato: Economia Digitale e Sociologia delle Reti

gruppo 8:

- codice identificativo del gruppo: G0588
- nome del gruppo: Gruppo Psicologia e Data Visualization
- insegnamento a cui il gruppo è dedicato: Psicologia Sociale e Data Visualization

Il gruppo viene poi collegato al corso o ai corsi a cui è dedicato tramite una relazione del grafo.

- ❖ gestione delle relazioni sociali e accademiche
 - uno studente può seguire uno o più corsi
 - un docente può insegnare in uno o più corsi
 - uno studente può conoscere altri studenti
 - uno studente può partecipare ad uno o più gruppi di studio
 - un gruppo di studio è dedicato a uno specifico corso
 - uno studente può interagire con uno o più docenti

esempio:

Il database a grafo deve rappresentare anche i collegamenti tra studenti, docenti, corsi e gruppi di studio.

- SEGUE: collega uno studente a un corso universitario
- INSEGNA: collega un docente a un corso universitario
- CONOSCE: collega uno studente a un altro studente
- PARTECIPA_A: collega uno studente a un gruppo di studio
- DEDICATO_A: collega un gruppo di studio a uno o più corsi
- INTERAGISCE_CON: collega uno studente a un docente

★ requisiti funzionali

Oltre alla memorizzazione dei dati, il sistema deve supportare alcune operazioni di consultazione e analisi. In particolare, deve essere possibile:

- **Trovare gli studenti collegati a un altro studente (query suggerita).**

Il sistema deve permettere di individuare gli studenti collegati a uno specifico studente attraverso relazioni di amicizia o partecipazione agli stessi gruppi di studio.

Questa funzionalità è utile per analizzare le reti sociali tra studenti e capire quali persone sono connesse tra loro direttamente o indirettamente.

Ad esempio, partendo da Luca Veraglia, il database può permettere di individuare gli studenti che conosce direttamente oppure quelli collegati a lui perchè partecipano allo stesso gruppo di studio.

- **Individuare i docenti collegati al maggior numero di studenti (query suggerita).**

Il sistema deve permettere di individuare quali docenti risultano collegati al maggior numero di studenti attraverso i corsi insegnati.

Questa funzionalità consente di analizzare il ruolo dei docenti all'interno della rete accademica e di capire quali insegnamenti coinvolgono più studenti.

Ad esempio, se un docente insegna un corso seguito da molti studenti, il database deve permettere di riconoscere che quel docente ha un ruolo importante nella rete accademica.

- **Individuare gli studenti più centrali nei gruppi di studio**

Il sistema deve consentire di individuare gli studenti che partecipano a più gruppi o che risultano collegati a un maggior numero di altri studenti.

Questa analisi è utile per riconoscere gli studenti più centrali nella rete di collaborazione universitaria.

Ad esempio, uno studente che partecipa sia al **Gruppo Big Data** sia al **Gruppo Statistica & Matematica** può risultare più centrale rispetto a uno studente che partecipa a un solo gruppo.

- **Scoprire connessioni indirette tra persone**

Il sistema deve permettere di scoprire collegamenti indiretti tra studenti, docenti e gruppi. Ad esempio, due studenti potrebbero non conoscersi direttamente, ma essere collegati perchè partecipano allo stesso gruppo di studio o seguono lo stesso corso.

Questa funzionalità rappresenta uno degli aspetti più importanti del modello a grafo, poiché permette di analizzare percorsi e connessioni che in un database relazionale sarebbero più difficili da individuare.

Ad esempio, due studenti potrebbero non conoscersi direttamente, ma essere collegati perchè seguono lo stesso corso, partecipano allo stesso gruppo di studio o interagiscono con lo stesso docente.

- **Individuare il corso seguito dal maggior numero di studenti**

Il sistema deve permettere di individuare quale corso universitario è seguito dal maggior numero di studenti.

Questa funzionalità consente di riconoscere gli insegnamenti che coinvolgono una parte maggiore della popolazione studentesca rappresentata nel database.

- **Individuare il gruppo di studio con il maggior numero di partecipanti**

Il sistema deve permettere di individuare quale gruppo di studio presenta il maggior numero di studenti partecipanti.

Questa analisi consente di riconoscere i gruppi maggiormente frequentati all'interno della rete universitaria.

- **Individuare gli studenti che seguono il maggior numero di corsi**

Il sistema deve consentire di individuare gli studenti collegati al maggior numero di corsi attraverso la relazione SEGUE.

Questa funzionalità permette di osservare quali studenti risultano maggiormente coinvolti nelle attività didattiche rappresentate nel grafo.

- **Individuare i docenti che insegnano più di un corso**

Il sistema deve permettere di individuare i docenti collegati a più di un corso attraverso la relazione INSEGNA.

Questa analisi consente di riconoscere i docenti responsabili di più insegnamenti all'interno della rete accademica.

Individuazione dei nodi

Dopo aver completato l'analisi dei requisiti, è possibile individuare i principali nodi che compongono il database a grafo del social network universitario.

In un database a grafo, i nodi rappresentano gli elementi principali del sistema, cioè gli oggetti o le persone di cui si vogliono memorizzare le informazioni. Ogni nodo possiede alcune proprietà che permettono di descriverlo e identificarlo.

In questa fase viene utilizzato il **modello a grafo di Neo4j** per individuare gli elementi principali del social network universitario. Nel modello a grafo le informazioni principali vengono rappresentate tramite **nodi**.

I nodi rappresentano gli elementi fondamentali del contesto universitario, cioè studenti, docenti, corsi e gruppi di studio. Ogni nodo possiede delle **proprietà**, utili a descriverlo e identificarlo.

In questa fase non viene ancora scritto il codice definitivo, ma viene definita la struttura logica del grafo che successivamente sarà implementata in **Neo4j** tramite il linguaggio **Cypher**.

I nodi principali individuati sono:

- Studente
- Docente
- Corso
- GruppoStudio

❖ Nodo studente

questo rappresenta una persona iscritta all'università. Ogni studente può seguire corsi, partecipare a gruppi di studio, conoscere altri studenti e interagire con i docenti.

Esempio:

```
Studente {  
  matricola: 20066579,  
  nome: "Luca",  
  cognome: "Veraglia",  
  corsoLaurea: "Data Science",  
  annoIscrizione: 2024  
}
```

La proprietà più importante è la matricola, perchè permette di identificare in modo univoco ogni studente all'interno del database.

❖ Nodo docente

Il nodo docente rappresenta un professore universitario, questo può essere collegato ai corsi che insegna e agli studenti con cui interagisce.

Esempio:


```
Docente {  
  nome: "Gianluca",  
  cognome: "Bellini",  
  settoreDisciplinare: "Informatica",  
  email: "Gianluca.bellini@unisalento.it"  
}
```

La proprietà **email** può essere utilizzata come identificativo, perchè rappresenta un contatto istituzionale univoco.

❖ Nodo corso

Il nodo corso rappresenta un insegnamento universitario. Ogni corso può essere seguito da più studenti ed essere insegnato da uno o più docenti.

Esempio:

```
Corso {  
  codiceCorso: "B089",  
  nome: "Analisi dei dati e Big Data",  
  crediti: 8  
}
```

La proprietà **codiceCorso** permette di identificare in modo univoco ogni corso.

❖ Nodo gruppo studio

Il nodo **GruppoStudio** rappresenta un gruppo formato da studenti che collaborano nello studio di uno o più insegnamenti.

Esempio:

```
GruppoStudio {  
  codiceGruppo: "G009",  
  nome: "Gruppo Big Data"  
}
```

Analisi delle relazioni

In questa fase viene utilizzato il modello a grafo di Neo4j per analizzare i collegamenti tra i nodi individuati nella fase precedente.

Nel database a grafo del social network universitario, le relazioni rappresentano i legami sociali e accademici tra studenti, docenti, corsi e gruppi di studio. Inoltre, nel modello a grafo le relazioni sono elementi espliciti e permettono di attraversare facilmente il grafo.

Le principali relazioni individuate sono:

```
(:Studente)-[:SEGUE]->(:Corso)  
(:Docente)-[:INSEGNA]->(:Corso)  
(:Studente)-[:CONOSCE]->(:Studente)  
(:Studente)-[:PARTECIPA_A]->(:GruppoStudio)  
(:GruppoStudio)-[:DEDICATO_A]->(:Corso)  
(:Studente)-[:INTERAGISCE_CON]->(:Docente)
```

- Relazione **SEGUE**: collega uno studente a un corso universitario. Serve a rappresentare gli insegnamenti frequentati dagli studenti.
Alcuni esempi:

Luca Veraglia SEGUE Analisi dei dati e Big Data
 Luca Veraglia SEGUE Machine Learning
 Luca Veraglia SEGUE Programmazione Python

Marta De Stradis SEGUE Internet of Things
 Marta De Stradis SEGUE Basi di Dati
 Marta De Stradis SEGUE Analisi Matematica

Patrizia Triponea SEGUE Statistica
 Patrizia Triponea SEGUE Analisi Matematica
 Patrizia Triponea SEGUE Basi di Dati

Paolo Riccardini SEGUE Sociologia delle Reti
 Paolo Riccardini SEGUE Psicologia Sociale
 Paolo Riccardini SEGUE Analisi dei dati e Big Data

Elena Moretti SEGUE Economia Digitale
 Elena Moretti SEGUE Statistica
 Elena Moretti SEGUE Lingua Inglese

Francesca Marino SEGUE Psicologia Sociale
 Francesca Marino SEGUE Sociologia delle Reti
 Francesca Marino SEGUE Data Visualization

Matteo Leone SEGUE Internet of Things
 Matteo Leone SEGUE Basi di Dati
 Matteo Leone SEGUE Programmazione Python

Roberto Longo SEGUE Statistica
 Roberto Longo SEGUE Analisi dei dati e Big Data
 Roberto Longo SEGUE Data Visualization

- Relazione INSEGNA: collega un docente ad un corso universitario. Serve a rappresentare quali insegnamenti sono tenuti dai diversi docenti.
 Esempi:

Gianluca Bellini INSEGNA Analisi dei dati e Big Data

Alessandro Rinaldi INSEGNA Lingua Inglese

Giulia Ferraro INSEGNA Statistica
 Giulia Ferraro INSEGNA Analisi Matematica

Martina De Santis INSEGNA Internet of Things

Paolo Marchetti INSEGNA Machine Learning
 Paolo Marchetti INSEGNA Data Visualization

Laura Conti INSEGNA Economia Digitale

Stefano Greco INSEGNA Sociologia delle Reti
 Stefano Greco INSEGNA Psicologia Sociale

Elena Romano INSEGNA Basi di Dati
 Elena Romano INSEGNA Programmazione Python

Questa relazione permette di collegare indirettamente i docenti agli studenti attraverso i corsi frequentati.

- Relazione CONOSCE: collega uno studente a un altro studente. Serve a rappresentare i rapporti sociali diretti tra studenti.
 Alcuni esempi:

Luca Veraglia CONOSCE Marta De Stradis
 Luca Veraglia CONOSCE Sara Romano

Luca Veraglia CONOSCE Chiara Ferri

Marta De Stradis CONOSCE Patrizia Triponea
Marta De Stradis CONOSCE Davide Greco
Marta De Stradis CONOSCE Andrea Rizzo

Paolo Riccardini CONOSCE Marco Santoro
Paolo Riccardini CONOSCE Nicola Esposito

Sara Romano CONOSCE Chiara Ferri
Sara Romano CONOSCE Alessia Costa

Elena Moretti CONOSCE Simone De Luca
Elena Moretti CONOSCE Martina Serra
Giulia Conti CONOSCE Valentina Gallo

Francesca Marino CONOSCE Valentina Gallo
Federico Caruso CONOSCE Roberto Longo

- Relazione PARTECIPA_A: questa collega uno studente a un gruppo studio. Serve a rappresentare la partecipazione degli studenti ai gruppi collegati agli insegnamenti.
Esempi:

Luca Veraglia PARTECIPA_A Gruppo Big Data
Luca Veraglia PARTECIPA_A Gruppo Machine Learning
Luca Veraglia PARTECIPA_A Gruppo Database e Python

Marta De Stradis PARTECIPA_A Gruppo Internet
Marta De Stradis PARTECIPA_A Gruppo Statistica e Matematica
Marta De Stradis PARTECIPA_A Gruppo Database e Python

Paolo Riccardini PARTECIPA_A Gruppo Big Data
Paolo Riccardini PARTECIPA_A Gruppo Economia e Reti
Paolo Riccardini PARTECIPA_A Gruppo Psicologia e Data Visualization

Elena Moretti PARTECIPA_A Gruppo Studio Inglese
Elena Moretti PARTECIPA_A Gruppo Economia e Reti
Elena Moretti PARTECIPA_A Gruppo Psicologia e Data Visualization

Matteo Leone PARTECIPA_A Gruppo Internet
Matteo Leone PARTECIPA_A Gruppo Machine Learning
Matteo Leone PARTECIPA_A Gruppo Database e Python

Roberto Longo PARTECIPA_A Gruppo Big Data
Roberto Longo PARTECIPA_A Gruppo Statistica e Matematica
Roberto Longo PARTECIPA_A Gruppo Psicologia e Data Visualization

- Relazione DEDICATO_A: collega un gruppo studio al corso o ai corsi a cui è dedicato
Un gruppo può essere dedicato a uno o più corsi. Questa scelta consente di rappresentare anche gruppi interdisciplinari, dedicati a insegnamenti collegati tra loro.

Esempi:

Gruppo Big Data DEDICATO_A Analisi dei dati e Big Data

Gruppo Statistica e Matematica DEDICATO_A Statistica
Gruppo Statistica e Matematica DEDICATO_A Analisi Matematica

Gruppo Internet DEDICATO_A Internet of Things

Gruppo Studio Inglese DEDICATO_A Lingua Inglese

Gruppo Machine Learning DEDICATO_A Machine Learning

Gruppo Database e Python DEDICATO_A Basi di Dati
Gruppo Database e Python DEDICATO_A Programmazione Python

Gruppo Economia e Reti DEDICATO_A Economia Digitale
Gruppo Economia e Reti DEDICATO_A Sociologia delle Reti

Gruppo Psicologia e Data Visualization DEDICATO_A Psicologia Sociale
Gruppo Psicologia e Data Visualization DEDICATO_A Data Visualization

- Relazione INTERAGISCE_CON: collega uno studente ad un docente. Serve a rappresentare le interazioni didattiche, richieste di chiarimento, comunicazioni e attività legate ai corsi.
Esempi:

Luca Veraglia INTERAGISCE_CON Gianluca Bellini
Luca Veraglia INTERAGISCE_CON Paolo Marchetti
Luca Veraglia INTERAGISCE_CON Elena Romano

Marta De Stradis INTERAGISCE_CON Martina De Santis
Marta De Stradis INTERAGISCE_CON Elena Romano
Marta De Stradis INTERAGISCE_CON Gianluca Bellini

Paolo Riccardini INTERAGISCE_CON Stefano Greco
Paolo Riccardini INTERAGISCE_CON Alessandro Rinaldi
Paolo Riccardini INTERAGISCE_CON Gianluca Bellini

Elena Moretti INTERAGISCE_CON Laura Conti
Elena Moretti INTERAGISCE_CON Giulia Ferraro
Elena Moretti INTERAGISCE_CON Alessandro Rinaldi

Francesca Marino INTERAGISCE_CON Stefano Greco
Francesca Marino INTERAGISCE_CON Alessandro Rinaldi
Francesca Marino INTERAGISCE_CON Paolo Marchetti

Roberto Longo INTERAGISCE_CON Giulia Ferraro
Roberto Longo INTERAGISCE_CON Gianluca Bellini
Roberto Longo INTERAGISCE_CON Paolo Marchetti

Nel modello a grafo le relazioni sono rappresentate con una direzione, utile per indicare il verso del collegamento tra i nodi. Ad esempio, la relazione SEGUE parte dal nodo Studente e arriva al nodo Corso, perché è lo studente che segue un determinato insegnamento.

Le relazioni individuate possono avere cardinalità diverse. Uno studente può seguire più corsi e uno stesso corso può essere seguito da più studenti; quindi la relazione SEGUE è di tipo molti-a-molti. Allo stesso modo, uno studente può partecipare a più gruppi di studio e un gruppo può includere più studenti. La relazione CONOSCE, invece, collega due nodi dello stesso tipo, cioè due studenti, e rappresenta un rapporto sociale diretto.

Questa struttura permette di rappresentare in modo flessibile sia i collegamenti accademici sia quelli sociali, rendendo possibile l'analisi delle connessioni dirette e indirette tra i nodi del grafo.

Le relazioni individuate permettono di costruire una rete universitaria composta da collegamenti sociali e accademici. Attraverso queste relazioni sarà possibile eseguire query per trovare studenti collegati tra loro, individuare docenti centrali, analizzare gruppi di studio e scoprire connessioni indirette tra persone.

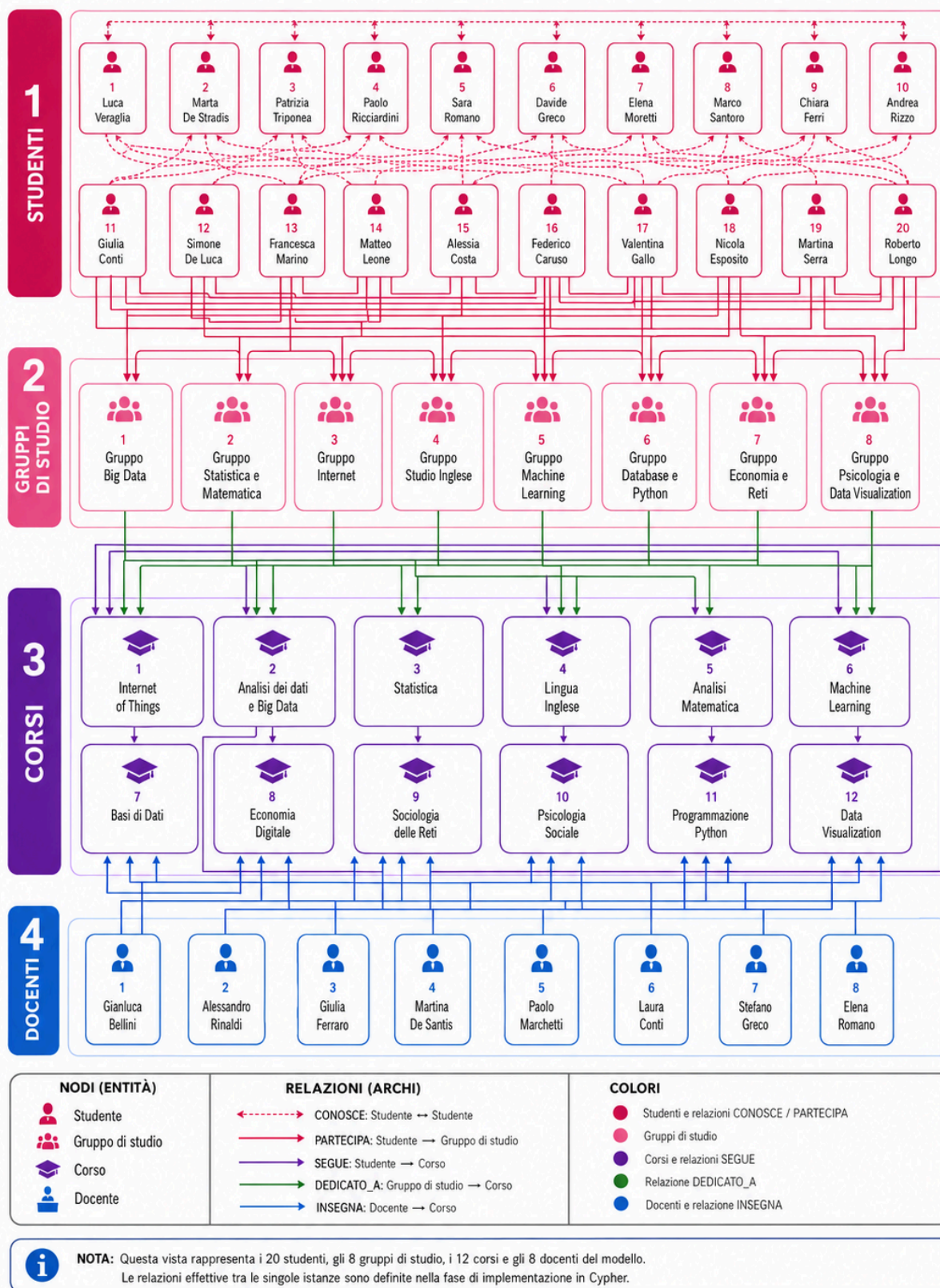
Dopo aver individuato i nodi principali e analizzato le relazioni tra essi, si procede con la progettazione complessiva del modello a grafo. In questa fase viene utilizzato il modello a grafo di **Neo4j**, che consente di rappresentare il contesto universitario attraverso nodi collegati tra loro da relazioni esplicite. I nodi individuati nella fase precedente, cioè studenti, docenti, corsi e gruppi di studio, vengono organizzati in una rete che permette di descrivere sia i collegamenti accademici sia quelli sociali.

Il modello progettato consente di rappresentare situazioni reali del contesto universitario. Ad esempio, uno studente può essere collegato a un corso perché lo segue, a un gruppo di studio perché vi partecipa, ad altri studenti perché li conosce e a un docente perché interagisce con lui. Allo stesso tempo, un docente può essere collegato ai corsi che insegna e, indirettamente, agli studenti che frequentano quei corsi.

Questa struttura permette di costruire una rete universitaria in cui ogni elemento non viene considerato in modo isolato, ma in relazione agli altri elementi presenti nel grafo. In questo modo è possibile analizzare percorsi, collegamenti indiretti, gruppi di collaborazione e nodi più centrali all'interno della rete.

Rispetto a un database relazionale, in cui i collegamenti tra i dati vengono ricostruiti attraverso tabelle e operazioni di join, nel modello a grafo le relazioni sono già parte della struttura del database. Questo rende più immediata l'analisi delle connessioni tra studenti, docenti, corsi e gruppi di studio.

Social network universitario - modello a grafo



La figura precedente mostra il modello concettuale del grafo.

Implementazione dell'ambiente di lavoro con Docker e Neo4j

Dopo aver progettato il modello a grafo si procede con la preparazione dell'ambiente di lavoro necessario per implementare concretamente il database. In questa fase vengono utilizzati Docker e Neo4j. Docker consente di avviare neo4j all'interno di un container, cioè un ambiente isolato e replicabile facilmente. In questo modo non è necessario installare manualmente tutti i componenti del database sul computer ma è possibile avviarli attraverso un file di configurazione.

Neo4j viene utilizzato come database a grafo, perché permette di rappresentare i dati attraverso nodi, proprietà e relazioni. In questo caso, viene utilizzato per creare i nodi *Studente*, *Docente*, *Corso* e *GruppoStudio*, e per collegarli attraverso relazioni come *SEGUE*, *CONOSCE*, *PARTECIPA_A*, *INSEGNA*, *DEDICATO_A* e *INTERAGISCE_CON*.

Vengono inoltre utilizzati: Neo4j Browser per visualizzare il grafo e creare le query e Cypher per la fase successiva così da creare nodi e relazioni.

L'ambiente di lavoro può essere organizzato in una cartella di progetto contenente i file principali:

```
social-network-universitario/
|
|— docker-compose.yml
|— creazione_grafo.cypher
|— query_grafo.cypher
|— main.py
```

Il file `docker-compose.yml` serve per avviare Neo4j tramite Docker.

Il file `creazione_grafo.cypher` contiene le istruzioni per creare i nodi e le relazioni del grafo.

Il file `query_grafo.cypher` contiene le query utilizzate per interrogare il database.

Il file `main.py` verrà utilizzato successivamente per collegare Python a Neo4j ed eseguire alcune interrogazioni in modo automatico.

Per avviare Neo4j con Docker può essere utilizzato il seguente file di configurazione:

```
services:
  neo4j:
    image: neo4j:latest
    container_name: neo4j_social_network
    ports:
      - "7474:7474"
      - "7687:7687"
    environment:
      - NEO4J_AUTH=neo4j/password123
    volumes:
      - neo4j_data:/data

volumes:
  neo4j_data:
```

In questo file viene indicata l'immagine Docker di Neo4j da utilizzare. La porta **7474** permette di accedere a **Neo4j Browser**, cioè l'interfaccia grafica usata per visualizzare e interrogare il grafo. La porta **7687**, invece, permette la connessione al database tramite altri strumenti, come Python.

La variabile `NEO4J_AUTH` permette di impostare le credenziali di accesso al database. In questo esempio, l'utente è `neo4j` e la password è `password123`.

Per avviare Neo4j, dal terminale bisogna posizionarsi nella cartella del progetto ed eseguire il comando:

```
docker compose up -d
```

Il comando avvia il container in background. Dopo l'avvio, Neo4j può essere aperto dal browser all'indirizzo:

```
http://localhost:7474
```

Creazione e popolamento del grafo con Cypher

Dopo aver configurato l'ambiente di lavoro con Docker e Neo4j, si procede con la creazione concreta del database a grafo.

In questa fase viene utilizzato Cypher, il linguaggio query di Neo4j. Cypher consente di creare nodi, assegnare proprietà e definire relazioni tra gli elementi del grafo.

In questo progetto il codice Cypher viene scritto ed eseguito direttamente all'interno di **Neo4j Browser**, l'interfaccia grafica di Neo4j. Attraverso Neo4j Browser è possibile creare i nodi, definire le relazioni e visualizzare graficamente il risultato del database a grafo.

La creazione del grafo prevede due passaggi:

1. prima avviene la creazione dei nodi: studenti, docenti, corsi e gruppi di studio.
2. dopodichè vengono definite le relazioni: SEGUE, INSEGNA, CONOSCE, PARTECIPA_A, DEDICATO_A, INTERAGISCE_CON.

I dati utilizzati sono quelli individuati nelle fasi precedenti del progetto, cioè studenti, docenti, corsi e gruppi di studio già definiti nei requisiti informativi.

Il grafo è composto complessivamente da:

- 20 nodi Studente;
- 8 nodi Docente;
- 12 nodi Corso;
- 8 nodi Gruppo Studio.

La prima parte del codice riguarda la creazione dei nodi principali del grafo. Ogni nodo rappresenta un elemento del social network universitario.

I nodi creati sono:

- Studente
- Docente
- Corso
- GruppoStudio

Per ogni nodo vengono inserite alcune proprietà. Ad esempio, per gli studenti vengono memorizzati matricola, nome, cognome, corso di laurea e anno di iscrizione. Per i docenti vengono memorizzati nome, cognome, settore disciplinare ed email. Per i corsi vengono inseriti codice, nome e crediti, mentre per i gruppi di studio vengono inseriti codice identificativo e nome del gruppo.

La creazione e il popolamento del grafo vengono svolti in tre momenti principali:

1. definizione dei vincoli di unicità;
2. creazione dei nodi e delle relative proprietà;
3. creazione delle relazioni tra i nodi.

- definizione dei vincoli di unicità

Prima di inserire i dati vengono definiti alcuni vincoli di unicità. Questi vincoli permettono di mantenere il database coerente, impedendo la creazione di più nodi con lo stesso identificativo.

Per ciascuna categoria di nodo viene individuata una proprietà univoca:

- la matricola per gli studenti;
- l'indirizzo email per i docenti;
- il codice del corso per i corsi;
- il codice identificativo per i gruppi di studio.

I vincoli vengono creati mediante le seguenti istruzioni Cypher:

```
CREATE CONSTRAINT studente_matricola IF NOT EXISTS
FOR (s:Studente)
REQUIRE s.matricola IS UNIQUE;

CREATE CONSTRAINT docente_email IF NOT EXISTS
FOR (d:Docente)
REQUIRE d.email IS UNIQUE;

CREATE CONSTRAINT corso_codice IF NOT EXISTS
FOR (c:Corso)
REQUIRE c.codiceCorso IS UNIQUE;

CREATE CONSTRAINT gruppo_codice IF NOT EXISTS
FOR (g:GruppoStudio)
REQUIRE g.codiceGruppo IS UNIQUE;
```

La clausola IF NOT EXISTS crea il vincolo soltanto quando non è già presente nel database. In questo modo, le istruzioni possono essere eseguite nuovamente senza generare errori dovuti alla presenza dello stesso vincolo.

- creazione nodi studente

La prima categoria di nodi inserita nel grafo è Studente.

Per ogni studente vengono memorizzate le seguenti proprietà:

- matricola;
- nome;
- cognome;
- corso di laurea;
- anno di iscrizione.

Per inserire più studenti viene utilizzata l'istruzione UNWIND, che trasforma la lista dei dati in singole righe da elaborare.

L'istruzione MERGE verifica se un nodo con la stessa matricola è già presente. Se non esiste, il nodo viene creato; se esiste già, le sue proprietà vengono aggiornate senza produrre duplicati.

```
UNWIND [
{
  matricola: "20066579",
  nome: "Luca",
  cognome: "Veraglia",
  corsoLaurea: "Data Science",
  annoIscrizione: 2024
```

```

},
{
  matricola: "20088965",
  nome: "Marta",
  cognome: "De Stradis",
  corsoLaurea: "Ingegneria Informatica",
  annoIscrizione: 2024
},
{
  matricola: "20029863",
  nome: "Patrizia",
  cognome: "Triponea",
  corsoLaurea: "Ingegneria dell'Informazione",
  annoIscrizione: 2023
},
{
  matricola: "20016894",
  nome: "Paolo",
  cognome: "Riccardini",
  corsoLaurea: "Sociologia",
  annoIscrizione: 2025
},
{
  matricola: "20073421",
  nome: "Sara",
  cognome: "Romano",
  corsoLaurea: "Data Science",
  annoIscrizione: 2023
},
{
  matricola: "20054782",
  nome: "Davide",
  cognome: "Greco",
  corsoLaurea: "Ingegneria Informatica",
  annoIscrizione: 2022
},
{
  matricola: "20091234",
  nome: "Elena",
  cognome: "Moretti",
  corsoLaurea: "Economia",
  annoIscrizione: 2024
},
{
  matricola: "20045671",
  nome: "Marco",
  cognome: "Santoro",
  corsoLaurea: "Sociologia",
  annoIscrizione: 2023
},
{
  matricola: "20082345",
  nome: "Chiara",
  cognome: "Ferri",
  corsoLaurea: "Data Science",
  annoIscrizione: 2025
},
{
  matricola: "20037519",
  nome: "Andrea",
  cognome: "Rizzo",
  corsoLaurea: "Ingegneria dell'Informazione",
  annoIscrizione: 2024
},
{
  matricola: "20069124",
  nome: "Giulia",
  cognome: "Conti",

```

```

corsoLaurea: "Lingue",
annoIscrizione: 2023
},
{
matricola: "20028467",
nome: "Simone",
cognome: "De Luca",
corsoLaurea: "Economia",
annoIscrizione: 2022
},
{
matricola: "20075643",
nome: "Francesca",
cognome: "Marino",
corsoLaurea: "Psicologia",
annoIscrizione: 2024
},
{
matricola: "20093418",
nome: "Matteo",
cognome: "Leone",
corsoLaurea: "Ingegneria Informatica",
annoIscrizione: 2025
},
{
matricola: "20051276",
nome: "Alessia",
cognome: "Costa",
corsoLaurea: "Data Science",
annoIscrizione: 2022
},
{
matricola: "20084329",
nome: "Federico",
cognome: "Caruso",
corsoLaurea: "Statistica",
annoIscrizione: 2023
},
{
matricola: "20036785",
nome: "Valentina",
cognome: "Gallo",
corsoLaurea: "Lingue",
annoIscrizione: 2024
},
{
matricola: "20097831",
nome: "Nicola",
cognome: "Esposito",
corsoLaurea: "Sociologia",
annoIscrizione: 2022
},
{
matricola: "20062145",
nome: "Martina",
cognome: "Serra",
corsoLaurea: "Economia",
annoIscrizione: 2025
},
{
matricola: "20048902",
nome: "Roberto",
cognome: "Longo",
corsoLaurea: "Statistica",
annoIscrizione: 2024
}
] AS studente

```

```
MERGE (s:Studente {matricola: studente.matricola})
SET s.nome = studente.nome,
    s.cognome = studente.cognome,
    s.corsoLaurea = studente.corsoLaurea,
    s.annoIscrizione = studente.annoIscrizione;
```

Al termine dell'esecuzione vengono creati 20 nodi appartenenti alla categoria Studente.

- creazione nodi Docente

Il nodo Docente rappresenta un professore universitario.

Per ogni docente vengono memorizzate le seguenti proprietà:

- nome;
- cognome;
- settore disciplinare;
- email istituzionale.

L'indirizzo email viene utilizzato come identificativo univoco.

```
UNWIND [
{
  nome: "Gianluca",
  cognome: "Bellini",
  settoreDisciplinare: "Informatica",
  email: "gianluca.bellini@unisalento.it"
},
{
  nome: "Alessandro",
  cognome: "Rinaldi",
  settoreDisciplinare: "Lingue",
  email: "alessandro.rinaldi@unisalento.it"
},
{
  nome: "Giulia",
  cognome: "Ferraro",
  settoreDisciplinare: "Statistica e Matematica",
  email: "giulia.ferraro@unisalento.it"
},
{
  nome: "Martina",
  cognome: "De Santis",
  settoreDisciplinare: "Internet of Things",
  email: "martina.desantis@unisalento.it"
},
{
  nome: "Paolo",
  cognome: "Marchetti",
  settoreDisciplinare: "Machine Learning",
  email: "paolo.marchetti@unisalento.it"
},
{
  nome: "Laura",
  cognome: "Conti",
  settoreDisciplinare: "Economia digitale",
  email: "laura.conti@unisalento.it"
}
```

```

},
{
  nome: "Stefano",
  cognome: "Greco",
  settoreDisciplinare: "Sociologia",
  email: "stefano.greco@unisalento.it"
},
{
  nome: "Elena",
  cognome: "Romano",
  settoreDisciplinare: "Basi di dati e sistemi informativi",
  email: "elena.romano@unisalento.it"
}
] AS docente

```

```

MERGE (d:Docente {email: docente.email})
SET d.nome = docente.nome,
    d.cognome = docente.cognome,
    d.settoreDisciplinare = docente.settoreDisciplinare;

```

Al termine dell'esecuzione vengono creati 8 nodi appartenenti alla categoria Docente.

- creazione nodi Corso

Il nodo Corso rappresenta un insegnamento universitario.

Per ogni corso vengono memorizzate le seguenti proprietà:

- codice del corso;
- nome;
- numero di crediti.

```

UNWIND [
{
  codiceCorso: "C001",
  nome: "Internet of Things",
  crediti: 9
},
{
  codiceCorso: "B089",
  nome: "Analisi dei dati e Big Data",
  crediti: 8
},
{
  codiceCorso: "A290",
  nome: "Statistica",
  crediti: 8
},
{
  codiceCorso: "C008",
  nome: "Lingua Inglese",
  crediti: 6
},
{
  codiceCorso: "A390",
  nome: "Analisi Matematica",
  crediti: 9
},
{
  codiceCorso: "B120",
  nome: "Machine Learning",
  crediti: 9
}
]

```

```

},
{
  codiceCorso: "C115",
  nome: "Basi di Dati",
  crediti: 8
},
{
  codiceCorso: "E210",
  nome: "Economia Digitale",
  crediti: 6
},
{
  codiceCorso: "S310",
  nome: "Sociologia delle Reti",
  crediti: 6
},
{
  codiceCorso: "P205",
  nome: "Psicologia Sociale",
  crediti: 8
},
{
  codiceCorso: "D330",
  nome: "Programmazione Python",
  crediti: 9
},
{
  codiceCorso: "M410",
  nome: "Data Visualization",
  crediti: 6
}
} AS corso

MERGE (c:Corso {codiceCorso: corso.codiceCorso})
SET c.nome = corso.nome,
    c.crediti = corso.crediti;

```

Al termine dell'esecuzione vengono creati 12 nodi appartenenti alla categoria Corso.

- creazione dei nodi GruppiStudio

Il nodo GruppoStudio rappresenta un gruppo formato da studenti che collaborano nello studio di uno o più insegnamenti.

Per ogni gruppo vengono memorizzati:

- codice identificativo;
- nome del gruppo.

L'insegnamento a cui il gruppo è dedicato non viene inserito come semplice proprietà. Il collegamento viene rappresentato successivamente attraverso la relazione DEDICATO_A tra il gruppo e il relativo corso.

```

UNWIND [
  {
    codiceGruppo: "G009",
    nome: "Gruppo Big Data"
  },
  {
    codiceGruppo: "G0863",
    nome: "Gruppo Statistica e Matematica"
  },
  {
    codiceGruppo: "G0108",
    nome: "Gruppo Internet"
  },
]

```

```

{
  codiceGruppo: "G6409",
  nome: "Gruppo Studio Inglese"
},
{
  codiceGruppo: "G0215",
  nome: "Gruppo Machine Learning"
},
{
  codiceGruppo: "G0320",
  nome: "Gruppo Database e Python"
},
{
  codiceGruppo: "G0477",
  nome: "Gruppo Economia e Reti"
},
{
  codiceGruppo: "G0588",
  nome: "Gruppo Psicologia e Data Visualization"
}
] AS gruppo

MERGE (g:GruppoStudio {codiceGruppo: gruppo.codiceGruppo})
SET g.nome = gruppo.nome;

```

Al termine della creazione dei nodi, il database contiene complessivamente 48 nodi.

- creazione relazioni sociali e accademiche

Dopo aver creato i nodi, si procede con la definizione delle relazioni sociali e accademiche.

Le relazioni inserite nel grafo sono:

SEGUE;
INSEGNA;
CONOSCE;
PARTECIPA_A;
DEDICATO_A;
INTERAGISCE_CON.

Anche per la creazione delle relazioni viene utilizzata l'istruzione UNWIND. Per ogni elemento della lista vengono individuati i nodi interessati tramite MATCH e viene poi creata la relazione attraverso MERGE.

- relazione segue:

La relazione SEGUE collega ogni studente ai corsi universitari frequentati.

```

UNWIND [
  {matricola: "20066579", codiceCorso: "B089"},
  {matricola: "20066579", codiceCorso: "A290"},
  {matricola: "20066579", codiceCorso: "B120"},
  {matricola: "20066579", codiceCorso: "D330"},
  {matricola: "20066579", codiceCorso: "M410"},

  {matricola: "20088965", codiceCorso: "C001"},
  {matricola: "20088965", codiceCorso: "A390"},
  {matricola: "20088965", codiceCorso: "C115"},
  {matricola: "20088965", codiceCorso: "D330"},
  {matricola: "20088965", codiceCorso: "B120"},

  {matricola: "20029863", codiceCorso: "A290"},
  {matricola: "20029863", codiceCorso: "A390"},
  {matricola: "20029863", codiceCorso: "C115"},
  {matricola: "20029863", codiceCorso: "C001"},
  {matricola: "20029863", codiceCorso: "D330"},

  {matricola: "20016894", codiceCorso: "S310"},
  {matricola: "20016894", codiceCorso: "P205"},

```

```

{matricola: "20016894", codiceCorso: "B089"},
{matricola: "20016894", codiceCorso: "C008"},

{matricola: "20073421", codiceCorso: "B089"},
{matricola: "20073421", codiceCorso: "A290"},
{matricola: "20073421", codiceCorso: "B120"},
{matricola: "20073421", codiceCorso: "M410"},
{matricola: "20073421", codiceCorso: "C115"},

{matricola: "20054782", codiceCorso: "C001"},
{matricola: "20054782", codiceCorso: "C115"},
{matricola: "20054782", codiceCorso: "D330"},
{matricola: "20054782", codiceCorso: "A390"},
{matricola: "20054782", codiceCorso: "B120"},

{matricola: "20091234", codiceCorso: "E210"},
{matricola: "20091234", codiceCorso: "A290"},
{matricola: "20091234", codiceCorso: "C008"},
{matricola: "20091234", codiceCorso: "M410"},

{matricola: "20045671", codiceCorso: "S310"},
{matricola: "20045671", codiceCorso: "P205"},
{matricola: "20045671", codiceCorso: "C008"},
{matricola: "20045671", codiceCorso: "B089"},

{matricola: "20082345", codiceCorso: "B089"},
{matricola: "20082345", codiceCorso: "A290"},
{matricola: "20082345", codiceCorso: "B120"},
{matricola: "20082345", codiceCorso: "D330"},
{matricola: "20082345", codiceCorso: "M410"},

{matricola: "20037519", codiceCorso: "C001"},
{matricola: "20037519", codiceCorso: "C115"},
{matricola: "20037519", codiceCorso: "D330"},
{matricola: "20037519", codiceCorso: "A390"},
{matricola: "20037519", codiceCorso: "B089"},

{matricola: "20069124", codiceCorso: "C008"},
{matricola: "20069124", codiceCorso: "P205"},
{matricola: "20069124", codiceCorso: "S310"},
{matricola: "20069124", codiceCorso: "E210"},

{matricola: "20028467", codiceCorso: "E210"},
{matricola: "20028467", codiceCorso: "A290"},
{matricola: "20028467", codiceCorso: "B089"},
{matricola: "20028467", codiceCorso: "M410"},

{matricola: "20075643", codiceCorso: "P205"},
{matricola: "20075643", codiceCorso: "S310"},
{matricola: "20075643", codiceCorso: "C008"},
{matricola: "20075643", codiceCorso: "M410"},

{matricola: "20093418", codiceCorso: "C001"},
{matricola: "20093418", codiceCorso: "C115"},
{matricola: "20093418", codiceCorso: "D330"},
{matricola: "20093418", codiceCorso: "B120"},
{matricola: "20093418", codiceCorso: "A390"},

{matricola: "20051276", codiceCorso: "B089"},
{matricola: "20051276", codiceCorso: "A290"},
{matricola: "20051276", codiceCorso: "B120"},
{matricola: "20051276", codiceCorso: "M410"},
{matricola: "20051276", codiceCorso: "D330"},

{matricola: "20084329", codiceCorso: "A290"},
{matricola: "20084329", codiceCorso: "A390"},
{matricola: "20084329", codiceCorso: "B089"},
{matricola: "20084329", codiceCorso: "M410"},

```



```

{matricola: "20084329", codiceCorso: "B120"},

{matricola: "20036785", codiceCorso: "C008"},
{matricola: "20036785", codiceCorso: "P205"},
{matricola: "20036785", codiceCorso: "S310"},
{matricola: "20036785", codiceCorso: "E210"},

{matricola: "20097831", codiceCorso: "S310"},
{matricola: "20097831", codiceCorso: "P205"},
{matricola: "20097831", codiceCorso: "B089"},
{matricola: "20097831", codiceCorso: "E210"},

{matricola: "20062145", codiceCorso: "E210"},
{matricola: "20062145", codiceCorso: "A290"},
{matricola: "20062145", codiceCorso: "B089"},
{matricola: "20062145", codiceCorso: "C008"},

{matricola: "20048902", codiceCorso: "A290"},
{matricola: "20048902", codiceCorso: "A390"},
{matricola: "20048902", codiceCorso: "B089"},
{matricola: "20048902", codiceCorso: "M410"},
{matricola: "20048902", codiceCorso: "D330"}
] AS relazione

MATCH (s:Studente {matricola: relazione.matricola})
MATCH (c:Corso {codiceCorso: relazione.codiceCorso})
MERGE (s)-[:SEGUE]->(c);

```

Vengono create complessivamente 91 relazioni SEGUE.

- relazione insegna:

La relazione INSEGNA collega ciascun docente ai corsi di cui è responsabile.

```

UNWIND [
{
  email: "gianluca.bellini@unisalento.it",
  codiceCorso: "B089"
},
{
  email: "alessandro.rinaldi@unisalento.it",
  codiceCorso: "C008"
},
{
  email: "giulia.ferraro@unisalento.it",
  codiceCorso: "A290"
},
{
  email: "giulia.ferraro@unisalento.it",
  codiceCorso: "A390"
},
{
  email: "martina.desantis@unisalento.it",
  codiceCorso: "C001"
},
{
  email: "paolo.marchetti@unisalento.it",
  codiceCorso: "B120"
},
{
  email: "paolo.marchetti@unisalento.it",
  codiceCorso: "M410"
},
{
  email: "laura.conti@unisalento.it",
  codiceCorso: "E210"
}
]

```

```

    },
    {
      email: "stefano.greco@unisalento.it",
      codiceCorso: "S310"
    },
    {
      email: "stefano.greco@unisalento.it",
      codiceCorso: "P205"
    },
    {
      email: "elena.romano@unisalento.it",
      codiceCorso: "C115"
    },
    {
      email: "elena.romano@unisalento.it",
      codiceCorso: "D330"
    }
  ] AS relazione

MATCH (d:Docente {email: relazione.email})
MATCH (c:Corso {codiceCorso: relazione.codiceCorso})
MERGE (d)-[:INSEGNA]->(c);

```

Vengono create complessivamente 12 relazioni INSEGNA.

- relazione conosce:

La relazione CONOSCE rappresenta una conoscenza diretta tra due studenti.

La relazione viene memorizzata con una direzione, ma nelle interrogazioni viene considerata reciproca e può quindi essere attraversata in entrambi i versi.

```

UNWIND [
  {matricola1: "20066579", matricola2: "20088965"},
  {matricola1: "20066579", matricola2: "20073421"},
  {matricola1: "20066579", matricola2: "20082345"},
  {matricola1: "20066579", matricola2: "20051276"},
  {matricola1: "20066579", matricola2: "20084329"},
  {matricola1: "20066579", matricola2: "20048902"},

  {matricola1: "20088965", matricola2: "20029863"},
  {matricola1: "20088965", matricola2: "20054782"},
  {matricola1: "20088965", matricola2: "20037519"},
  {matricola1: "20088965", matricola2: "20093418"},

  {matricola1: "20029863", matricola2: "20054782"},
  {matricola1: "20029863", matricola2: "20037519"},
  {matricola1: "20029863", matricola2: "20093418"},

  {matricola1: "20016894", matricola2: "20045671"},
  {matricola1: "20016894", matricola2: "20097831"},
  {matricola1: "20016894", matricola2: "20075643"},
  {matricola1: "20016894", matricola2: "20069124"},

  {matricola1: "20073421", matricola2: "20082345"},
  {matricola1: "20073421", matricola2: "20051276"},
  {matricola1: "20073421", matricola2: "20084329"},

  {matricola1: "20054782", matricola2: "20037519"},
  {matricola1: "20054782", matricola2: "20093418"},

  {matricola1: "20091234", matricola2: "20028467"},
  {matricola1: "20091234", matricola2: "20062145"},
  {matricola1: "20091234", matricola2: "20069124"},

  {matricola1: "20045671", matricola2: "20097831"},

```

```

{matricola1: "20045671", matricola2: "20075643"},

{matricola1: "20082345", matricola2: "20051276"},
{matricola1: "20082345", matricola2: "20048902"},

{matricola1: "20037519", matricola2: "20093418"},

{matricola1: "20069124", matricola2: "20036785"},
{matricola1: "20069124", matricola2: "20075643"},

{matricola1: "20028467", matricola2: "20062145"},
{matricola1: "20028467", matricola2: "20097831"},

{matricola1: "20075643", matricola2: "20036785"},
{matricola1: "20075643", matricola2: "20097831"},

{matricola1: "20051276", matricola2: "20084329"},
{matricola1: "20051276", matricola2: "20048902"},

{matricola1: "20084329", matricola2: "20048902"},

{matricola1: "20036785", matricola2: "20062145"},

{matricola1: "20097831", matricola2: "20062145"}
] AS relazione

MATCH (s1:Studente {matricola: relazione.matricola1})
MATCH (s2:Studente {matricola: relazione.matricola2})
MERGE (s1)-[CONOSCE]->(s2);

```

Vengono create complessivamente 41 relazioni CONOSCE.

- relazione partecipa_a

La relazione PARTECIPA_A collega gli studenti ai gruppi di studio di cui fanno parte.

```

UNWIND [
{matricola: "20066579", codiceGruppo: "G009"},
{matricola: "20016894", codiceGruppo: "G009"},
{matricola: "20073421", codiceGruppo: "G009"},
{matricola: "20045671", codiceGruppo: "G009"},
{matricola: "20082345", codiceGruppo: "G009"},
{matricola: "20037519", codiceGruppo: "G009"},
{matricola: "20028467", codiceGruppo: "G009"},
{matricola: "20051276", codiceGruppo: "G009"},
{matricola: "20084329", codiceGruppo: "G009"},
{matricola: "20097831", codiceGruppo: "G009"},
{matricola: "20062145", codiceGruppo: "G009"},
{matricola: "20048902", codiceGruppo: "G009"},

{matricola: "20066579", codiceGruppo: "G0863"},
{matricola: "20088965", codiceGruppo: "G0863"},
{matricola: "20029863", codiceGruppo: "G0863"},
{matricola: "20073421", codiceGruppo: "G0863"},
{matricola: "20054782", codiceGruppo: "G0863"},
{matricola: "20091234", codiceGruppo: "G0863"},
{matricola: "20082345", codiceGruppo: "G0863"},
{matricola: "20037519", codiceGruppo: "G0863"},
{matricola: "20028467", codiceGruppo: "G0863"},
{matricola: "20093418", codiceGruppo: "G0863"},
{matricola: "20051276", codiceGruppo: "G0863"},
{matricola: "20084329", codiceGruppo: "G0863"},
{matricola: "20062145", codiceGruppo: "G0863"},
{matricola: "20048902", codiceGruppo: "G0863"},

{matricola: "20088965", codiceGruppo: "G0108"},

```

```

{matricola: "20029863", codiceGruppo: "G0108"},
{matricola: "20054782", codiceGruppo: "G0108"},
{matricola: "20037519", codiceGruppo: "G0108"},
{matricola: "20093418", codiceGruppo: "G0108"},

{matricola: "20016894", codiceGruppo: "G6409"},
{matricola: "20091234", codiceGruppo: "G6409"},
{matricola: "20045671", codiceGruppo: "G6409"},
{matricola: "20069124", codiceGruppo: "G6409"},
{matricola: "20075643", codiceGruppo: "G6409"},
{matricola: "20036785", codiceGruppo: "G6409"},
{matricola: "20062145", codiceGruppo: "G6409"},

{matricola: "20066579", codiceGruppo: "G0215"},
{matricola: "20088965", codiceGruppo: "G0215"},
{matricola: "20073421", codiceGruppo: "G0215"},
{matricola: "20054782", codiceGruppo: "G0215"},
{matricola: "20082345", codiceGruppo: "G0215"},
{matricola: "20093418", codiceGruppo: "G0215"},
{matricola: "20051276", codiceGruppo: "G0215"},
{matricola: "20084329", codiceGruppo: "G0215"},

{matricola: "20066579", codiceGruppo: "G0320"},
{matricola: "20088965", codiceGruppo: "G0320"},
{matricola: "20029863", codiceGruppo: "G0320"},
{matricola: "20073421", codiceGruppo: "G0320"},
{matricola: "20054782", codiceGruppo: "G0320"},
{matricola: "20082345", codiceGruppo: "G0320"},
{matricola: "20037519", codiceGruppo: "G0320"},
{matricola: "20093418", codiceGruppo: "G0320"},
{matricola: "20051276", codiceGruppo: "G0320"},
{matricola: "20048902", codiceGruppo: "G0320"},

{matricola: "20016894", codiceGruppo: "G0477"},
{matricola: "20091234", codiceGruppo: "G0477"},
{matricola: "20045671", codiceGruppo: "G0477"},
{matricola: "20069124", codiceGruppo: "G0477"},
{matricola: "20028467", codiceGruppo: "G0477"},
{matricola: "20075643", codiceGruppo: "G0477"},
{matricola: "20036785", codiceGruppo: "G0477"},
{matricola: "20097831", codiceGruppo: "G0477"},
{matricola: "20062145", codiceGruppo: "G0477"},

{matricola: "20066579", codiceGruppo: "G0588"},
{matricola: "20016894", codiceGruppo: "G0588"},
{matricola: "20073421", codiceGruppo: "G0588"},
{matricola: "20091234", codiceGruppo: "G0588"},
{matricola: "20082345", codiceGruppo: "G0588"},
{matricola: "20069124", codiceGruppo: "G0588"},
{matricola: "20028467", codiceGruppo: "G0588"},
{matricola: "20075643", codiceGruppo: "G0588"},
{matricola: "20051276", codiceGruppo: "G0588"},
{matricola: "20084329", codiceGruppo: "G0588"},
{matricola: "20036785", codiceGruppo: "G0588"},
{matricola: "20048902", codiceGruppo: "G0588"}
] AS relazione

MATCH (s:Studente {matricola: relazione.matricola})
MATCH (g:GruppoStudio {codiceGruppo: relazione.codiceGruppo})
MERGE (s)-[:PARTECIPA_A]->(g);

```

Vengono create complessivamente 77 relazioni PARTECIPA_A.

- relazione dedicato_a

La relazione DEDICATO_A collega ogni gruppo di studio al corso o ai corsi di riferimento.

```

UNWIND [
  {codiceGruppo: "G009", codiceCorso: "B089"},

  {codiceGruppo: "G0863", codiceCorso: "A290"},
  {codiceGruppo: "G0863", codiceCorso: "A390"},

  {codiceGruppo: "G0108", codiceCorso: "C001"},

  {codiceGruppo: "G6409", codiceCorso: "C008"},

  {codiceGruppo: "G0215", codiceCorso: "B120"},

  {codiceGruppo: "G0320", codiceCorso: "C115"},
  {codiceGruppo: "G0320", codiceCorso: "D330"},

  {codiceGruppo: "G0477", codiceCorso: "E210"},
  {codiceGruppo: "G0477", codiceCorso: "S310"},

  {codiceGruppo: "G0588", codiceCorso: "P205"},
  {codiceGruppo: "G0588", codiceCorso: "M410"}
] AS relazione

MATCH (g:GruppoStudio {codiceGruppo: relazione.codiceGruppo})
MATCH (c:Corso {codiceCorso: relazione.codiceCorso})
MERGE (g)-[:DEDICATO_A]->(c);

```

Vengono create complessivamente 12 relazioni DEDICATO_A.

- relazione interagisce_con

La relazione INTERAGISCE_CON rappresenta un contatto diretto tra uno studente e un docente.

L'interazione può indicare, ad esempio, una richiesta di chiarimento, un ricevimento, una comunicazione o un'attività didattica.

```

UNWIND [
  {
    matricola: "20066579",
    email: "gianluca.bellini@unisalento.it"
  },
  {
    matricola: "20066579",
    email: "paolo.marchetti@unisalento.it"
  },

  {
    matricola: "20088965",
    email: "martina.desantis@unisalento.it"
  },
  {
    matricola: "20088965",
    email: "elena.romano@unisalento.it"
  },

  {
    matricola: "20029863",
    email: "giulia.ferraro@unisalento.it"
  },
  {
    matricola: "20029863",
    email: "elena.romano@unisalento.it"
  },

  {
    matricola: "20016894",

```

```

email: "stefano.greco@unisalento.it"
},
{
  matricola: "20016894",
  email: "gianluca.bellini@unisalento.it"
},

{
  matricola: "20073421",
  email: "gianluca.bellini@unisalento.it"
},
{
  matricola: "20073421",
  email: "paolo.marchetti@unisalento.it"
},

{
  matricola: "20054782",
  email: "martina.desantis@unisalento.it"
},
{
  matricola: "20054782",
  email: "elena.romano@unisalento.it"
},

{
  matricola: "20091234",
  email: "laura.conti@unisalento.it"
},
{
  matricola: "20091234",
  email: "giulia.ferraro@unisalento.it"
},

{
  matricola: "20045671",
  email: "stefano.greco@unisalento.it"
},
{
  matricola: "20045671",
  email: "alessandro.rinaldi@unisalento.it"
},

{
  matricola: "20082345",
  email: "gianluca.bellini@unisalento.it"
},
{
  matricola: "20082345",
  email: "paolo.marchetti@unisalento.it"
},

{
  matricola: "20037519",
  email: "martina.desantis@unisalento.it"
},
{
  matricola: "20037519",
  email: "elena.romano@unisalento.it"
},

{
  matricola: "20069124",
  email: "alessandro.rinaldi@unisalento.it"
},
{
  matricola: "20069124",
  email: "stefano.greco@unisalento.it"
}

```

```

},

{
  matricola: "20028467",
  email: "laura.conti@unisalento.it"
},
{
  matricola: "20028467",
  email: "gianluca.bellini@unisalento.it"
},

{
  matricola: "20075643",
  email: "stefano.greco@unisalento.it"
},
{
  matricola: "20075643",
  email: "paolo.marchetti@unisalento.it"
},

{
  matricola: "20093418",
  email: "martina.desantis@unisalento.it"
},
{
  matricola: "20093418",
  email: "elena.romano@unisalento.it"
},

{
  matricola: "20051276",
  email: "gianluca.bellini@unisalento.it"
},
{
  matricola: "20051276",
  email: "paolo.marchetti@unisalento.it"
},

{
  matricola: "20084329",
  email: "giulia.ferraro@unisalento.it"
},
{
  matricola: "20084329",
  email: "gianluca.bellini@unisalento.it"
},

{
  matricola: "20036785",
  email: "alessandro.rinaldi@unisalento.it"
},
{
  matricola: "20036785",
  email: "stefano.greco@unisalento.it"
},

{
  matricola: "20097831",
  email: "stefano.greco@unisalento.it"
},
{
  matricola: "20097831",
  email: "laura.conti@unisalento.it"
},

{
  matricola: "20062145",
  email: "laura.conti@unisalento.it"
}

```

```

    },
    {
      matricola: "20062145",
      email: "giulia.ferraro@unisalento.it"
    },
    {
      matricola: "20048902",
      email: "giulia.ferraro@unisalento.it"
    },
    {
      matricola: "20048902",
      email: "paolo.marchetti@unisalento.it"
    }
  ] AS relazione

```

```

MATCH (s:Studente {matricola: relazione.matricola})
MATCH (d:Docente {email: relazione.email})
MERGE (s)-[:INTERAGISCE_CON]->(d);

```

Vengono create complessivamente 40 relazioni INTERAGISCE_CON.

- verifica del popolamento del grafo:

Dopo aver eseguito tutti i blocchi di codice, è possibile verificare il numero di nodi presenti nel database attraverso la seguente query:

```

MATCH (n)
RETURN labels(n)[0] AS tipoNodo,
       COUNT(n) AS numeroNodi
ORDER BY tipoNodo;

```

Il risultato è:

tipo di nodo	numero
corso	12
docente	8
gruppo studio	8
studente	20

Il database contiene quindi complessivamente 48 nodi.

Per verificare le relazioni viene utilizzata la seguente query:

```

MATCH ()-[r]->()
RETURN type(r) AS tipoRelazione,
       COUNT(r) AS numeroRelazioni
ORDER BY tipoRelazione;

```

Il risultato è:

tipo di relazione	numero
CONOSCE	41

DEDICATO_A	12
INSEGNA	12
INTERAGISCE_CON	40
PARTECIPA_A	77
SEGUE	91

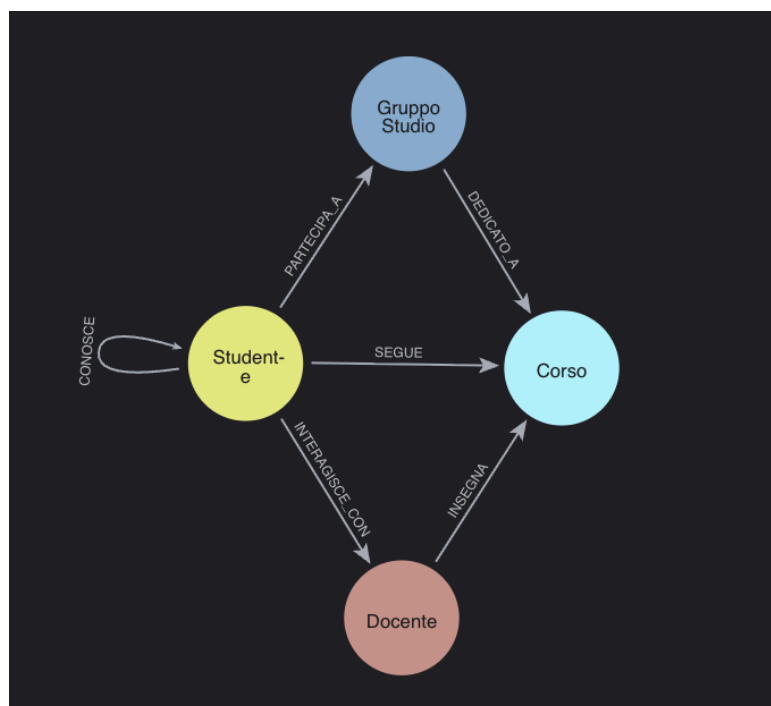
Il grafo contiene complessivamente 273 relazioni.

Una volta completato il popolamento, studenti, docenti, corsi e gruppi di studio non risultano più elementi isolati, ma formano una rete composta da collegamenti sociali e accademici.

Il database può quindi essere interrogato attraverso Cypher per analizzare le conoscenze tra studenti, la partecipazione ai gruppi, i corsi frequentati, i rapporti con i docenti e i percorsi diretti o indiretti presenti nella rete universitaria.

La corretta implementazione della struttura progettata può essere osservata anche attraverso lo schema generato direttamente da Neo4j.

La figura seguente mostra i quattro tipi principali di nodo presenti nel database, cioè Studente, Docente, Corso e GruppoStudio, e le relazioni che li collegano. La visualizzazione conferma che il modello concettuale definito nella fase di progettazione è stato correttamente implementato nel database.



La struttura del grafo permette di analizzare sia i collegamenti sociali sia quelli accademici. Attraverso le relazioni tra studenti e gruppi di studio è possibile osservare quali studenti risultano collegati tra loro, direttamente tramite conoscenze personali oppure indirettamente attraverso la partecipazione agli stessi gruppi.

Allo stesso modo, attraverso il collegamento tra docenti, corsi e studenti, è possibile individuare quali docenti risultano maggiormente connessi alla rete universitaria. In questo caso, il corso diventa il nodo intermedio che collega il docente agli studenti che lo seguono.

Questa organizzazione rende il modello utile non solo per memorizzare le informazioni, ma anche per interpretare le connessioni presenti all'interno del social network universitario.

Interrogazione del grafo con Cypher

In questa fase il database a grafo viene interrogato attraverso **Cypher**, il linguaggio di query utilizzato da Neo4j.

Le interrogazioni permettono di analizzare i collegamenti presenti nella rete universitaria e di ottenere informazioni che non sono state memorizzate direttamente come proprietà dei nodi, ma che possono essere ricavate attraversando le relazioni del grafo.

In questa fase vengono utilizzate principalmente le seguenti istruzioni:

- **MATCH**, per individuare nodi e relazioni che rispettano un determinato modello;
- **OPTIONAL MATCH**, per cercare collegamenti che potrebbero anche non essere presenti;
- **WHERE**, per applicare condizioni ai risultati;
- **WITH**, per elaborare e trasferire risultati intermedi tra diverse parti della query;
- **RETURN**, per specificare le informazioni da visualizzare;
- **COUNT**, per contare nodi o relazioni;
- **MAX**, per individuare il valore massimo tra quelli ottenuti;
- **collect**, per raccogliere più risultati all'interno di una lista;
- **UNWIND**, per trasformare gli elementi di una lista in singole righe;
- **DISTINCT**, per eliminare eventuali risultati duplicati;
- **ORDER BY**, per ordinare i risultati;
- **LIMIT**, per limitare il numero di righe restituite;
- **CASE**, per assegnare un valore diverso in base al tipo di collegamento individuato;
- **shortestPath**, per individuare il percorso più breve tra due nodi.
- **length**, per contare il numero di relazioni presenti in un percorso.

Prima di eseguire query più specifiche, è possibile visualizzare l'intero grafo creato, attraverso il seguente comando:

```
MATCH (n)-[r]->(m)
RETURN n, r, m;
```

La query ricerca tutte le relazioni presenti nel database.

- n rappresenta il nodo da cui parte la relazione
- r rappresenta la relazione
- m rappresenta il nodo a cui arriva la relazione

Risultato

L'esecuzione della query permette di visualizzare il grafo completo composto dai nodi relativi a studenti, docenti, corsi e gruppi di studio. In base ai dati inseriti, il grafo contiene:

Elemento	Quantità
Studenti	20
Docenti	8
Corsi	12
Gruppi di studio	8
Nodi totali	48

Relazioni totali	273
------------------	-----

- Studenti collegati tramite amicizie o gruppi di studio

La prima interrogazione permette di individuare gli studenti collegati a uno specifico studente attraverso due tipi di collegamento:

- conoscenza diretta, tramite la relazione **CONOSCE**;
- partecipazione allo stesso gruppo di studio, tramite la relazione **PARTECIPA_A**.

Ad esempio, partendo dallo studente **Luca Veraglia**, la query consente di trovare gli altri studenti collegati a lui.

```

MATCH (s:Studente {matricola: "20066579"})
OPTIONAL MATCH (s)-[:CONOSCE]-(amico:Studente)

WITH s,
      collect(DISTINCT amico) AS amici

OPTIONAL MATCH (s)-[:PARTECIPA_A]->
      (g:GruppoStudio)
      <-[:PARTECIPA_A]-
      (collega:Studente)

WHERE collega <> s

WITH amici,
      collect(DISTINCT collega) AS colleghi

UNWIND amici + colleghi AS studente

WITH DISTINCT studente,
      amici,
      colleghi

WHERE studente IS NOT NULL

RETURN studente.nome AS nome,
       studente.cognome AS cognome,
       studente.matricola AS matricola,
       CASE
         WHEN studente IN amici AND studente IN colleghi
         THEN "conoscenza diretta e gruppo comune"
         WHEN studente IN amici
         THEN "conoscenza diretta"
         ELSE "gruppo comune"
       END AS tipoCollegamento

ORDER BY cognome, nome;
```

Questa query permette di ricostruire la rete sociale e collaborativa di uno studente.

La prima istruzione:

```
MATCH (s:Studente {matricola: "20066579"})
```

individua Luca Veraglia attraverso la sua matricola.

Il primo **OPTIONAL MATCH** cerca gli studenti collegati direttamente a Luca mediante la relazione **CONOSCE**.

```
OPTIONAL MATCH (s)-[:CONOSCE]-(amico:Studente)
```

La relazione viene scritta senza freccia perché la conoscenza viene considerata reciproca e può essere cercata in entrambi i versi.

Gli studenti individuati vengono raccolti nella lista amici:
collect(DISTINCT amico) AS amici

Il secondo OPTIONAL MATCH individua gli studenti che partecipano ad almeno uno degli stessi gruppi di Luca:
(s)-[:PARTECIPA_A]->(g:GruppoStudio)

<-[:PARTECIPA_A]-(collega:Studente)

Il percorso analizzato è quindi:

Luca Veraglia

→ PARTECIPA_A

Gruppo di studio

← PARTECIPA_A

Altro studente

La condizione:

WHERE collega <> s

impedisce che Luca venga incluso tra gli studenti collegati a se stesso.

UNWIND trasforma la lista degli amici e quella dei colleghi di gruppo in singole righe. La clausola DISTINCT elimina gli studenti che compaiono più volte perché condividono diversi gruppi con Luca.

Infine, CASE stabilisce il tipo di collegamento:

- conoscenza diretta e gruppo comune;
- soltanto conoscenza diretta;
- soltanto gruppo comune.

Risultato

Partendo dallo studente Luca Veraglia, il database individua gli studenti collegati a lui tramite conoscenze dirette o partecipazione agli stessi gruppi di studio.

Nome	Cognome	Matricola	Tipo di collegamento
Federico	caruso	20084329	conoscenza diretta e gruppi comuni
Giulia	Conti	20069124	gruppo comune
Alessia	Costa	20051276	conoscenza diretta e gruppo comune
Simone	De Luca	20028467	gruppo comune
Marta	De Stradis	20088965	conoscenza diretta e gruppo comune
Nicola	Esposito	20097831	gruppo comune
Chiara	Ferri	20082345	conoscenza diretta e gruppo comune
Valentina	Gallo	20036785	gruppo comune
Davide	greco	20054782	gruppo comune
Matteo	Leone	20093418	gruppo comune
Roberto	Longo	20048902	conoscenza diretta e gruppo comune
Francesca	Marino	20075643	gruppo comune
Elena	Moretti	20091234	gruppo comune

Paolo	Riccardini	20016894	gruppo comune
Andrea	Rizzo	20037519	gruppo comune
Sara	Romano	20073421	conoscenza diretta e gruppo comune
Marco	Santoro	20045671	gruppo comune
Martina	Serra	20062145	gruppo comune
Patrizia	Triponea	20029863	gruppo comune

Luca Veraglia risulta collegato a tutti gli altri 19 studenti. Sei di essi sono collegati sia attraverso una conoscenza diretta sia attraverso almeno un gruppo comune, mentre gli altri sono raggiungibili attraverso la partecipazione agli stessi gruppi di studio.

La query permette quindi di ricostruire sia la rete sociale sia la rete collaborativa di uno studente.

- Docenti collegati al maggior numero di studenti

La seconda interrogazione permette di individuare quali docenti sono collegati al maggior numero di studenti attraverso i corsi insegnati.

Il percorso analizzato è il seguente:

Docente → INSEGNA → Corso ← SEGUE ← Studente

La query è:

```
MATCH (d:Docente)-[:INSEGNA]->(c:Corso)<-[:SEGUE]-(s:Studente)

RETURN d.nome AS nomeDocente,
       d.cognome AS cognomeDocente,
       collect(DISTINCT c.nome) AS corsiInsegnati,
       COUNT(DISTINCT s) AS numeroStudenti

ORDER BY numeroStudenti DESC,
       cognomeDocente,
       nomeDocente;
```

Questa query conta quanti studenti sono collegati a ciascun docente tramite i corsi che seguono. Il risultato viene ordinato dal docente con più studenti collegati a quello con meno studenti collegati.

L'istruzione: collect(DISTINCT c.nome) raccoglie in una lista i corsi insegnati da ogni docente.

L'istruzione: COUNT(DISTINCT s) conta il numero di studenti distinti collegati al docente.

DISTINCT è necessario perché uno stesso studente può seguire più corsi insegnati dallo stesso docente. Senza DISTINCT, lo studente verrebbe contato più volte.

Il risultato viene ordinato dal docente collegato al maggior numero di studenti a quello collegato al numero minore.

Risultato:

Docente	Corso/i insegnato/i	Numero studenti collegati
Giulia Ferraro	Statistica, analisi matematica	14

Gianluca Bellini	Analisi dei dati e Big Data	12
Paolo Marchetti	Machine Learning; Data Visualization;	12
Elena Romano	Basi di dati; Programmazione python	10
Alessandro Rinaldi	Lingua Inglese	7
Laura Conti	Economia Digitale	6
Stefano Greco	Sociologia delle Reti; Psicologia Sociale;	6
Martina De Santis	Internet of Things	5

La docente collegata al maggior numero di studenti risulta **Giulia Ferraro**, perché insegna i corsi di Statistica e Analisi Matematica, seguiti complessivamente da 14 studenti distinti.

- Corso seguito dal maggior numero di studenti

Questa query permette di individuare quale corso universitario è seguito dal maggior numero di studenti.

Il percorso analizzato è il seguente:

Studente -> SEGUE -> Corso

la query è:

```
MATCH (s:Studente)-[:SEGUE]->(c:Corso)
RETURN c.codiceCorso AS codiceCorso,
       c.nome AS nomeCorso,
       c.crediti AS crediti,
       COUNT(DISTINCT s) AS numeroStudenti
ORDER BY numeroStudenti DESC,
       nomeCorso
LIMIT 1;
```

La prima parte della query individua gli studenti collegati ai corsi universitari attraverso la relazione SEGUE.

L'istruzione: COUNT(DISTINCT s) AS numeroStudenti

conta il numero di studenti distinti che seguono ciascun corso. L'utilizzo di DISTINCT evita che uno stesso studente possa essere contato più volte.

L'istruzione: ORDER BY numeroStudenti DESC

ordina i corsi partendo da quello seguito dal maggior numero di studenti.

Infine: LIMIT 1

limita il risultato al primo corso della classifica, cioè quello con il maggior numero di studenti.

Risultato:

Codice Corso	Nome del Corso	Crediti	Numero studenti
B089	Analisi dei dati e Big Data	8	12

In base ai dati inseriti, il corso seguito dal maggior numero di studenti è Analisi dei dati e Big Data, identificato dal codice B089. Il corso assegna 8 crediti ed è seguito complessivamente da 12 studenti.

Questa interrogazione consente di individuare l'insegnamento con la maggiore partecipazione all'interno della rete accademica.

- Gruppo di studio con il maggior numero di partecipanti

Questa query permette di individuare il gruppo di studio a cui partecipa il maggior numero di studenti.

Il percorso analizzato è il seguente:

Studente → PARTECIPA_A → GruppoStudio

La query è:

```
MATCH (s:Studente)-[:PARTECIPA_A]->(g:GruppoStudio)
RETURN g.codiceGruppo AS codiceGruppo,
       g.nome AS nomeGruppo,
       COUNT(DISTINCT s) AS numeroPartecipanti
ORDER BY numeroPartecipanti DESC,
       nomeGruppo
LIMIT 1;
```

La prima istruzione: MATCH (s:Studente)-[:PARTECIPA_A]->(g:GruppoStudio) individua gli studenti collegati ai gruppi di studio attraverso la relazione PARTECIPA_A.

L'istruzione: COUNT(DISTINCT s) AS numeroPartecipanti conta il numero di studenti distinti che partecipano a ciascun gruppo. L'utilizzo di DISTINCT garantisce che ogni studente venga contato una sola volta all'interno dello stesso gruppo.

L'istruzione: ORDER BY numeroPartecipanti DESC ordina i gruppi di studio partendo da quello con il maggior numero di partecipanti.

Infine: LIMIT 1 limita il risultato al primo gruppo della classifica.

Risultato:

Codice gruppo	Nome del gruppo	Numero partecipanti
G0863	Gruppo Statistica e Matematica	14

In base ai dati inseriti, il gruppo di studio con il maggior numero di partecipanti è il Gruppo Statistica e Matematica, identificato dal codice G0863. Il gruppo è composto complessivamente da 14 studenti.

Questa interrogazione consente di individuare il gruppo che presenta il livello più elevato di partecipazione all'interno della rete universitaria.

- Studenti che seguono il maggior numero di corsi

Questa query permette di individuare gli studenti che seguono il maggior numero di corsi universitari.

Il percorso analizzato è il seguente:

Studente → SEGUE → Corso

La query è:

```
MATCH (s:Studente)-[:SEGUE]->(c:Corso)
WITH s, COUNT(DISTINCT c) AS numeroCorsi
WITH collect({
  studente: s,
  numeroCorsi: numeroCorsi
}) AS risultati,
MAX(numeroCorsi) AS massimoCorsi
UNWIND risultati AS risultato
WITH risultato.studente AS s,
risultato.numeroCorsi AS numeroCorsi,
massimoCorsi
WHERE numeroCorsi = massimoCorsi
RETURN s.nome AS nome,
s.cognome AS cognome,
s.matricola AS matricola,
numeroCorsi
ORDER BY cognome,
nome;
```

L'istruzione: MATCH (s:Studente)-[:SEGUE]->(c:Corso)
individua gli studenti collegati ai corsi universitari attraverso la relazione SEGUE.

L'istruzione: COUNT(DISTINCT c) AS numeroCorsi
conta il numero di corsi distinti seguiti da ciascuno studente. L'utilizzo di DISTINCT evita che uno stesso corso possa essere contato più volte per il medesimo studente.

L'istruzione: MAX(numeroCorsi) AS massimoCorsi
individua il numero più alto di corsi seguito da uno studente.

L'istruzione collect raccoglie temporaneamente gli studenti e il relativo numero di corsi, mentre UNWIND trasforma nuovamente i risultati raccolti in singole righe.

La condizione: WHERE numeroCorsi = massimoCorsi
mantiene soltanto gli studenti il cui numero di corsi corrisponde al valore massimo individuato.

In questo modo vengono restituiti tutti gli studenti a pari merito, senza limitare il risultato a un solo nominativo.

Risultato:

Nome	Cognome	Matricola	Numero corsi
Federico	Caruso	20084329	5
Alessia	Costa	20051276	5
Marta	De Stradis	20088965	5
Chiara	Ferri	20082345	5
Davide	Greco	20054782	5
Matteo	Leone	20093418	5
Roberto	Longo	20048902	5

Andrea	Rizzo	20037519	5
Sara	Romano	20073421	5
Patrizia	Triponea	20029863	5
Luca	Veraglia	20066579	5

In base ai dati inseriti, il numero massimo di corsi seguito da uno studente è pari a cinque. Questo valore viene raggiunto da undici studenti.

Questa interrogazione consente di individuare gli studenti con la maggiore partecipazione alle attività didattiche rappresentate all'interno della rete accademica.

- Docenti che insegnano più di un corso:

Questa query permette di individuare docenti che insegnano più di un corso universitario.

Il percorso analizzato è il seguente:

Docente → INSEGNA → Corso

La query è:

```
MATCH (d:Docente)-[:INSEGNA]->(c:Corso)
WITH d,
      COUNT(DISTINCT c) AS numeroCorsi,
      collect(DISTINCT c.nome) AS corsiInsegnati
WHERE numeroCorsi > 1
RETURN d.nome AS nome,
       d.cognome AS cognome,
       d.email AS email,
       numeroCorsi,
       corsiInsegnati
ORDER BY numeroCorsi DESC,
       cognome,
       nome;
```

L'istruzione: MATCH (d:Docente)-[:INSEGNA]->(c:Corso)
individua i docenti collegati ai corsi universitari attraverso la relazione INSEGNA.

L'istruzione: COUNT(DISTINCT c) AS numeroCorsi
conta il numero di corsi distinti insegnati da ciascun docente. L'utilizzo di DISTINCT evita che uno stesso corso possa essere contato più volte per il medesimo docente.

L'istruzione: collect(DISTINCT c.nome) AS corsiInsegnati
raccolge in una lista i nomi dei corsi insegnati da ciascun docente.

La condizione: WHERE numeroCorsi > 1
seleziona soltanto i docenti che insegnano almeno due corsi, escludendo quelli collegati a un solo insegnamento.

Infine, i risultati vengono ordinati in base al numero di corsi insegnati e, in caso di parità, in ordine alfabetico per cognome e nome.

Risultato:

Nome	Cognome	Email	Numero di corsi	Corsi insegnati
Giulia	Ferraro	giulia.ferraro@unisa	2	Statistica e analisi

		lento.it		matematica
Stefano	Greco	stefano.greco@unis alento.it	2	Sociologia delle reti e psicologia sociale
Paolo	Marchetti	paolo.marchetti@un isalento.it	2	Machine learning e data visualization
Elena	Romano	elena.romano@unis alento.it	2	Basi di dati e programmazione di python

In base ai dati inseriti, quattro docenti insegnano più di un corso universitario. Tutti i docenti individuati insegnano due corsi.

Questa interrogazione consente di individuare i docenti responsabili di più insegnamenti all'interno della rete accademica.

- Studenti più centrali nei gruppi studio

Questa query permette di individuare gli studenti più centrali nella rete dei gruppi di studio. Uno studente risulta più centrale quando partecipa a più gruppi o quando è collegato a un numero maggiore di altri studenti attraverso gruppi comuni.

```
MATCH (s:Studente)-[:PARTECIPA_A]->(g:GruppoStudio)
OPTIONAL MATCH (g)-[:PARTECIPA_A]-(altro:Studente)
WHERE s <> altro
RETURN s.nome AS nome,
       s.cognome AS cognome,
       COUNT(DISTINCT g) AS numeroGruppi,
       COUNT(DISTINCT altro) AS studentiCollegatiTramiteGruppi
ORDER BY numeroGruppi DESC, studentiCollegatiTramiteGruppi DESC,
       cognome,
       nome;
```

La prima parte della query individua i gruppi di studio frequentati da ciascuno studente.

COUNT(DISTINCT g)

conta il numero di gruppi diversi a cui partecipa lo studente.

La seconda parte cerca gli altri studenti iscritti agli stessi gruppi.

COUNT(DISTINCT altro)

conta il numero complessivo di studenti distinti raggiungibili attraverso almeno un gruppo comune.

Uno studente che condivide più gruppi con la stessa persona non la conta più volte, grazie all'utilizzo di **DISTINCT**.

Questa interrogazione è utile per osservare quali studenti hanno una posizione più attiva nella rete collaborativa universitaria.

Risultato:

Nome	Cognome	Numero gruppi	Studenti collegati tramite gruppi
Alessia	Costa	5	19

Chiara	Ferri	5	19
Sara	Romano	5	19
Luca	Veraglia	5	19
Federico	Caruso	4	19
Simone	De Luca	4	19
Roberto	Longo	4	19
Elena	Moretti	4	19
Martina	Serra	4	19
Andrea	Rizzo	4	16
Paolo	Riccardini	4	15
Marta	De Stradis	4	13
Davide	Greco	4	13
Matteo	Leone	4	13
Marco	Santoro	3	15
Giulia	Conti	3	14
Valentina	Gallo	3	14
Francesca	Marino	3	14
Patrizia	Triponea	3	13
Nicola	Esposito	2	15

Questa query consente di trovare quali studenti occupano una posizione più attiva nella rete collaborativa universitaria.

In base ai dati inseriti, gli studenti più centrali nei gruppi di studio risultano Alessia Costa, Chiara Ferri, Sara Romano e Luca Veraglia in quanto ognuno di essi partecipa a cinque gruppi studio ed è collegato, attraverso tali gruppi, a tutti gli altri 19 studenti presenti nel database.

- Connessioni indirette tra studenti e docenti

Il modello a grafo permette anche di individuare collegamenti indiretti tra nodi che non sono connessi direttamente.

Come esempio può essere cercato il percorso più breve tra:

- lo studente Luca Veraglia, identificato con la matricola 20066579.
- la docente Martina De Santis, identificata dall'email: martina.desantis@unisalento.it

Tra i due non è presente una relazione diretta INTERAGISCE_CON.

La query è la seguente:

```
MATCH percorso = shortestPath(
(s:Studente {matricola: "20066579"})-[*..5]-
(d:Docente {email: "martina.desantis@unisalento.it"})
```

```

)
RETURN
[n IN nodes(percorso) |
CASE
  WHEN n:Studente THEN n.nome + " " + n.cognome
  WHEN n:Docente THEN n.nome + " " + n.cognome
  ELSE n.nome
END
] AS percorso,
[r IN relationships(percorso) | type(r)] AS relazioni,
length(percorso) AS numeroPassaggi;

```

La funzione:

shortestPath()

ricerca il percorso formato dal minor numero possibile di relazioni tra i due nodi.

Il pattern:

-[*]-

indica che il percorso può attraversare un numero variabile di relazioni, senza limitarne il tipo.

L'assenza delle frecce permette di attraversare le relazioni in entrambi i versi.

L'istruzione:

length(percorso)

restituisce il numero di relazioni che compongono il percorso individuato.

Risultato:

Il percorso più breve è:

```

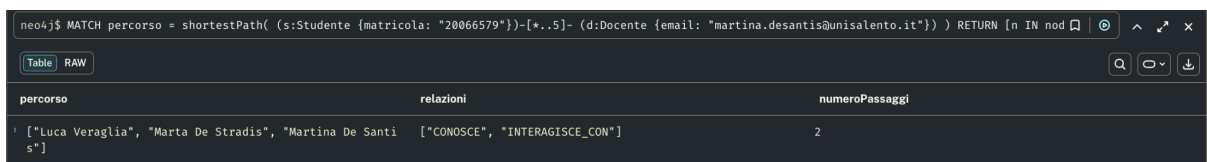
Luca Veraglia
  → CONOSCE →
Marta De Stradis
  → INTERAGISCE_CON →
Martina De Santis

```

Il percorso è quindi composto da due relazioni:

- Luca Veraglia conosce Marta De Stradis.
- Marta De Stradis interagisce con Martina De Santis.

Il valore restituito da Length(percorso) è quindi: 2. Questo mostra che Luca Veraglia e Martina De Santis non sono collegati direttamente, ma risultano connessi attraverso Marta De Stradis.



The screenshot shows a Neo4j Cypher query execution interface. The query is: `MATCH percorso = shortestPath( (s:Studente {matricola: "20066579"})-[*..5]- (d:Docente {email: "martina.desantis@unisalento.it"}) ) RETURN [n IN nod(percorso) | n.nome + " " + n.cognome] AS percorso, [r IN relationships(percorso) | type(r)] AS relazioni, length(percorso) AS numeroPassaggi`. The results are displayed in a table with three columns: **percorso**, **relazioni**, and **numeroPassaggi**. The first row shows the path from Luca Veraglia to Martina De Santis via Marta De Stradis, with the relationships **CONOSCE** and **INTERAGISCE_CON**, and a count of 2.

percorso	relazioni	numeroPassaggi
["Luca Veraglia", "Marta De Stradis", "Martina De Santis"]	["CONOSCE", "INTERAGISCE_CON"]	2

- riepilogo finale query cypher

Le query Cypher permettono di interrogare il database a grafo e di analizzare i collegamenti presenti nella rete universitaria.

Attraverso le interrogazioni eseguite è stato possibile osservare:

- individuare gli studenti collegati attraverso conoscenze dirette o gruppi di studio comuni (relazioni sociali e partecipazione ai gruppi di studio);

- individuare i docenti collegati al maggior numero di studenti attraverso i corsi insegnati (rapporti accademici tra studenti, corsi e docenti);
- riconoscere il corso seguito dal maggior numero di studenti;
- individuare il gruppo di studio con il maggior numero di partecipanti;
- riconoscere gli studenti che seguono il maggior numero di corsi;
- individuare i docenti che insegnano più di un corso;
- analizzare gli studenti più centrali nella rete dei gruppi di studio (posizione degli studenti nella rete collaborativa);
- scoprire connessioni indirette tra studenti e docenti (percorsi indiretti tra nodi non collegati direttamente).

Il database non viene quindi utilizzato soltanto per memorizzare i dati, ma anche per analizzare e interpretare le connessioni presenti nel social network universitario.

Utilizzo di Python per interrogare Neo4j

Dopo la creazione e l'interrogazione del grafo tramite Neo4j Browser, viene utilizzato anche **Python** per collegarsi al database ed eseguire alcune query in modo automatico.

In questa fase si lavora con:

- **Python**: per eseguire il codice
- **Neo4j Driver**: per collegare Python al database Neo4j
- **Neo4j**: come database a grafo già avviato tramite Docker

In questa fase Python non viene utilizzato per progettare il grafo, ma per collegarsi a Neo4j ed eseguire automaticamente alcune interrogazioni. Questo permette di rendere l'analisi più flessibile, perché le query possono essere salvate in uno script ed eseguite più volte senza doverle riscrivere manualmente nell'interfaccia grafica.

L'utilizzo di Python è utile soprattutto quando si vogliono automatizzare le interrogazioni, stampare i risultati in modo ordinato oppure integrare il database con altri strumenti di analisi.

Per collegare Python a Neo4j viene utilizzata la libreria ufficiale neo4j, che permette di stabilire una connessione con il database a grafo.

La libreria può essere installata tramite il comando:

```
pip install neo4j
```

- **connessione al database**

Il collegamento tra Python e Neo4j avviene attraverso l'indirizzo:

```
bolt://localhost:7687
```

La porta **7687** viene utilizzata da Neo4j per consentire la connessione da strumenti esterni, come Python.

Il seguente codice permette di stabilire la connessione al database:

```
from neo4j import GraphDatabase

uri = "bolt://localhost:7687"
username = "neo4j"
password = "password123"

driver = GraphDatabase.driver(uri, auth=(username, password))
```

In questo codice vengono specificati l'indirizzo del database e le credenziali di accesso, ossia lo username e la password definiti nel file [docker-compose.yml](#).

L'oggetto driver gestisce il collegamento tra lo script Python e Neo4j. La connessione viene utilizzata quando viene aperta una sessione ed eseguita una query.

Interrogazione 1: Studenti collegati a Luca Veraglia

Il primo esempio permette di trovare gli studenti collegati a **Luca Veraglia** tramite:

- una relazione diretta CONOSCE.
- la partecipazione ad almeno uno stesso gruppo di studio.

La query utilizzata è la stessa presentata nella sezione dedicata all'interrogazione del grafo con Cypher.

```
from neo4j import GraphDatabase

uri = "bolt://localhost:7687"
username = "neo4j"
password = "password123"

driver = GraphDatabase.driver(
    uri,
    auth=(username, password)
)

query = """
MATCH (s:Studente {matricola: "20066579"})

OPTIONAL MATCH (s)-[:CONOSCE]-(amico:Studente)

WITH s,
    collect(DISTINCT amico) AS amici

OPTIONAL MATCH (s)-[:PARTECIPA_A]->
    (g:GruppoStudio)
    <-[:PARTECIPA_A]-
    (collega:Studente)

WHERE collega <> s

WITH amici,
    collect(DISTINCT collega) AS colleghi

UNWIND amici + colleghi AS studente

WITH DISTINCT studente,
    amici,
    colleghi

WHERE studente IS NOT NULL

RETURN studente.nome AS nome,
    studente.cognome AS cognome,
    studente.matricola AS matricola,
    CASE
        WHEN studente IN amici AND studente IN colleghi
            THEN "conoscenza diretta e gruppo comune"
        WHEN studente IN amici
            THEN "conoscenza diretta"
        ELSE "gruppo comune"
    END AS tipoCollegamento

ORDER BY cognome, nome
"""

with driver.session() as session:
    risultati = session.run(query)
```

```
print("Studenti collegati a Luca Veraglia:")
```

```
for record in risultati:
```

```
    print(
        record["nome"],
        record["cognome"],
        "_",
        record["matricola"],
        "_",
        record["tipoCollegamento"]
    )
```

```
driver.close()
```

L'istruzione: `with driver.session()` as session:

apre una sessione di comunicazione con Neo4j. La sessione viene chiusa automaticamente al termine del blocco.

L'istruzione: `risultati = session.run(query)`

invia la query Cypher al database e restituisce i risultati prodotti da Neo4j.

Il ciclo: `for record in risultati:`

legge una riga alla volta e stampa il nome, il cognome, la matricola e il tipo di collegamento individuato.

Al termine viene eseguita l'istruzione: `driver.close()`

che chiude il driver e libera le risorse utilizzate.

Risultato:

Studenti collegati a Luca Veraglia:

Federico Caruso - 20084329 - conoscenza diretta e gruppo comune
Giulia Conti - 20069124 - gruppo comune
Alessia Costa - 20051276 - conoscenza diretta e gruppo comune
Simone De Luca - 20028467 - gruppo comune
Marta De Stradis - 20088965 - conoscenza diretta e gruppo comune
Nicola Esposito - 20097831 - gruppo comune
Chiara Ferri - 20082345 - conoscenza diretta e gruppo comune
Valentina Gallo - 20036785 - gruppo comune
Davide Greco - 20054782 - gruppo comune
Matteo Leone - 20093418 - gruppo comune
Roberto Longo - 20048902 - conoscenza diretta e gruppo comune
Francesca Marino - 20075643 - gruppo comune
Elena Moretti - 20091234 - gruppo comune
Paolo Riccardini - 20016894 - gruppo comune
Andrea Rizzo - 20037519 - gruppo comune
Sara Romano - 20073421 - conoscenza diretta e gruppo comune
Marco Santoro - 20045671 - gruppo comune
Martina Serra - 20062145 - gruppo comune
Patrizia Triponea - 20029863 - gruppo comune

```

(base) monkey10@Air-di-malika social-network-universitario % python3 studenti_luca.py
Studenti collegati a Luca Veraglia:
Federico Caruso - 20084329 - conoscenza diretta e gruppo comune
Giulia Conti - 20069124 - gruppo comune
Alessia Costa - 20051276 - conoscenza diretta e gruppo comune
Simone De Luca - 20028467 - gruppo comune
Marta De Stradis - 20088965 - conoscenza diretta e gruppo comune
Nicola Esposito - 20097831 - gruppo comune
Chiara Ferri - 20082345 - conoscenza diretta e gruppo comune
Valentina Gallo - 20036785 - gruppo comune
Davide Greco - 20054782 - gruppo comune
Matteo Leone - 20093418 - gruppo comune
Roberto Longo - 20048902 - conoscenza diretta e gruppo comune
Francesca Marino - 20075643 - gruppo comune
Elena Moretti - 20091234 - gruppo comune
Paolo Riccardini - 20016894 - gruppo comune
Andrea Rizzo - 20037519 - gruppo comune
Sara Romano - 20073421 - conoscenza diretta e gruppo comune
Marco Santoro - 20045671 - gruppo comune
Martina Serra - 20062145 - gruppo comune
Patrizia Triponea - 20029863 - gruppo comune

```

Il risultato mostra che Luca Veraglia è collegato a tutti gli altri 19 studenti presenti nel database.

Alcuni studenti sono collegati a Luca sia attraverso una relazione diretta di conoscenza sia attraverso la partecipazione agli stessi gruppi di studio. Gli altri sono collegati attraverso uno o più gruppi comuni.

L'esempio mostra come Python possa recuperare automaticamente i risultati della query e stamparli in una forma leggibile nel terminale.

Interrogazione 2: Docenti collegati al maggior numero di studenti

Il secondo esempio permette di individuare quali docenti sono collegati al maggior numero di studenti attraverso i corsi insegnati.

Il percorso analizzato è il seguente:

```

Docente → INSEGNA → Corso ← SEGUE ← Studente

```

Il codice utilizzato è:

```

from neo4j import GraphDatabase

uri = "bolt://localhost:7687"
username = "neo4j"
password = "password123"

driver = GraphDatabase.driver(
    uri,
    auth=(username, password)
)

query = """
MATCH (d:Docente)
    -[:INSEGNA]->
    (c:Corso)
    <-[:SEGUE]-
    (s:Studente)

RETURN d.nome AS nomeDocente,
    d.cognome AS cognomeDocente,
    collect(DISTINCT c.nome) AS corsiInsegnati,

```



```

COUNT(DISTINCT s) AS numeroStudenti

ORDER BY numeroStudenti DESC,
        cognomeDocente,
        nomeDocente
"""

with driver.session() as session:
    risultati = session.run(query)

    print(
        "Docenti collegati al maggior numero di studenti:"
    )

    for record in risultati:
        corsi = ", ".join(record["corsiInsegnati"])

        print(
            record["nomeDocente"],
            record["cognomeDocente"],
            "_",
            corsi,
            "_",
            record["numeroStudenti"],
            "studenti"
        )

driver.close()

```

La query parte dai nodi Docente, attraversa la relazione INSEGNA e raggiunge i corsi. Successivamente individua gli studenti che seguono tali corsi attraverso la relazione SEGUE.

L'istruzione Cypher: `collect(DISTINCT c.nome) AS corsiInsegnati` raccoglie i corsi insegnati da ciascun docente.

L'istruzione: `COUNT(DISTINCT s) AS numeroStudenti` conta il numero di studenti distinti collegati a ogni docente.

L'utilizzo di `DISTINCT` evita che uno stesso studente venga contato più volte quando segue più corsi insegnati dallo stesso docente.

Nel codice Python, l'istruzione: `corsi = ", ".join(record["corsiInsegnati"])` trasforma la lista dei corsi restituita da Neo4j in una singola stringa, separando i nomi con una virgola.

Risultato:

```

Docenti collegati al maggior numero di studenti:

Giulia Ferraro - Statistica, Analisi Matematica - 14 studenti
Gianluca Bellini - Analisi dei dati e Big Data - 12 studenti
Paolo Marchetti - Machine Learning, Data Visualization - 12 studenti
Elena Romano - Basi di Dati, Programmazione Python - 10 studenti
Alessandro Rinaldi - Lingua Inglese - 7 studenti
Laura Conti - Economia Digitale - 6 studenti
Stefano Greco - Sociologia delle Reti, Psicologia Sociale - 6 studenti
Martina De Santis - Internet of Things - 5 studenti

```

```
(base) monkey10@Air-di-malika social-network-universitario % python3 docenti_studenti.py

Docenti collegati al maggior numero di studenti:
Giulia Ferraro - Statistica, Analisi Matematica - 14 studenti
Gianluca Bellini - Analisi dei dati e Big Data - 12 studenti
Paolo Marchetti - Data Visualization, Machine Learning - 12 studenti
Elena Romano - Programmazione Python, Basi di Dati - 10 studenti
Alessandro Rinaldi - Lingua Inglese - 7 studenti
Laura Conti - Economia Digitale - 6 studenti
Stefano Greco - Sociologia delle Reti, Psicologia Sociale - 6 studenti
Martina De Santis - Internet of Things - 5 studenti
```

Il risultato mostra che **Giulia Ferraro** è la docente collegata al maggior numero di studenti.

La docente insegna i corsi di Statistica e Analisi Matematica, attraverso i quali risulta collegata complessivamente a 14 studenti distinti. Seguono Gianluca Bellini e Paolo Marchetti, entrambi collegati a 12 studenti.

L'utilizzo di Python consente di interrogare Neo4j in modo automatico e di ottenere risultati leggibili direttamente dal terminale. In questo progetto Python viene usato come strumento di supporto per eseguire query sul grafo e analizzare i collegamenti tra studenti, docenti, corsi e gruppi di studio.

In questo modo il database a grafo non viene consultato solo tramite Neo4j Browser, ma anche attraverso uno script esterno, rendendo l'analisi più flessibile e ripetibile.

Conclusioni

Il progetto ha permesso di progettare e realizzare un database a grafo per rappresentare un social network universitario, mettendo al centro non soltanto i dati dei singoli elementi, ma soprattutto i collegamenti tra studenti, docenti, corsi e gruppi di studio.

La scelta del modello a grafo risulta adatta a questo contesto perché consente di rappresentare in modo diretto le relazioni sociali e accademiche presenti all'interno dell'università. Studenti, docenti, corsi e gruppi di studio non vengono quindi considerati come elementi separati, ma come nodi appartenenti a un'unica rete collegata.

Attraverso Neo4j e il linguaggio Cypher è stato possibile creare e popolare il grafo, definendo i nodi, le relative proprietà e le relazioni tra gli elementi del modello. Le otto interrogazioni di analisi sviluppate hanno permesso di individuare gli studenti collegati attraverso conoscenze dirette o gruppi comuni, i docenti collegati al maggior numero di studenti, il corso più seguito, il gruppo di studio con più partecipanti, gli studenti che seguono più corsi, i docenti responsabili di più insegnamenti, gli studenti più centrali nella rete dei gruppi di studio e i percorsi indiretti tra nodi non collegati direttamente.

Dal punto di vista tecnico, Docker permette di configurare e avviare Neo4j all'interno di un ambiente isolato e riproducibile, facilitando l'esecuzione del progetto anche su dispositivi e sistemi operativi differenti. Python consente invece di collegarsi al database da uno script esterno, eseguire automaticamente le query Cypher e visualizzare i risultati direttamente dal terminale.

Il progetto mostra quindi come un database a grafo possa essere utilizzato non soltanto per memorizzare informazioni, ma anche per analizzare le relazioni e i percorsi presenti nella rete. Questa caratteristica rende Neo4j una soluzione efficace per rappresentare e studiare un contesto universitario nel quale i collegamenti tra gli elementi assumono un ruolo centrale.